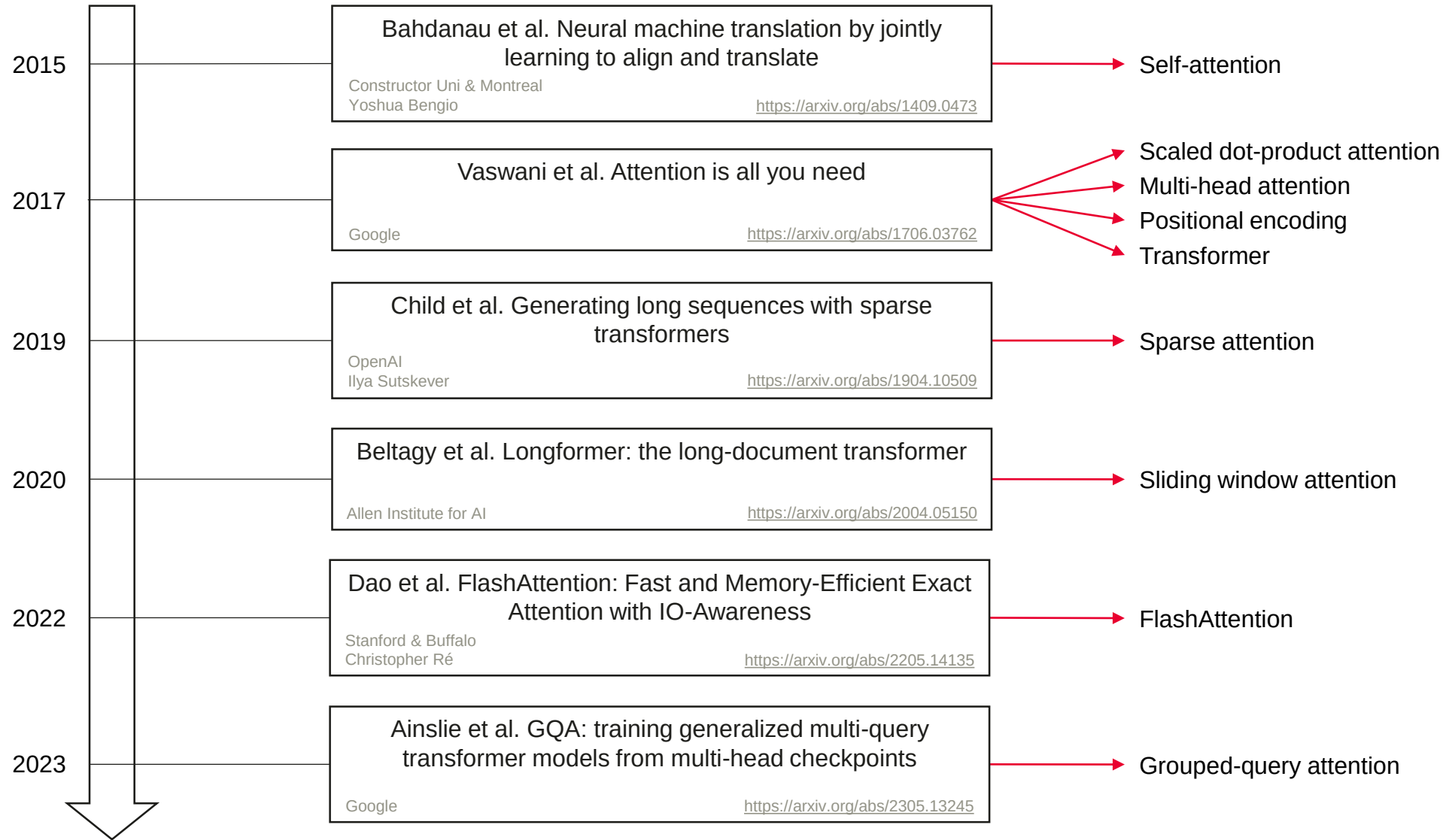


Attention

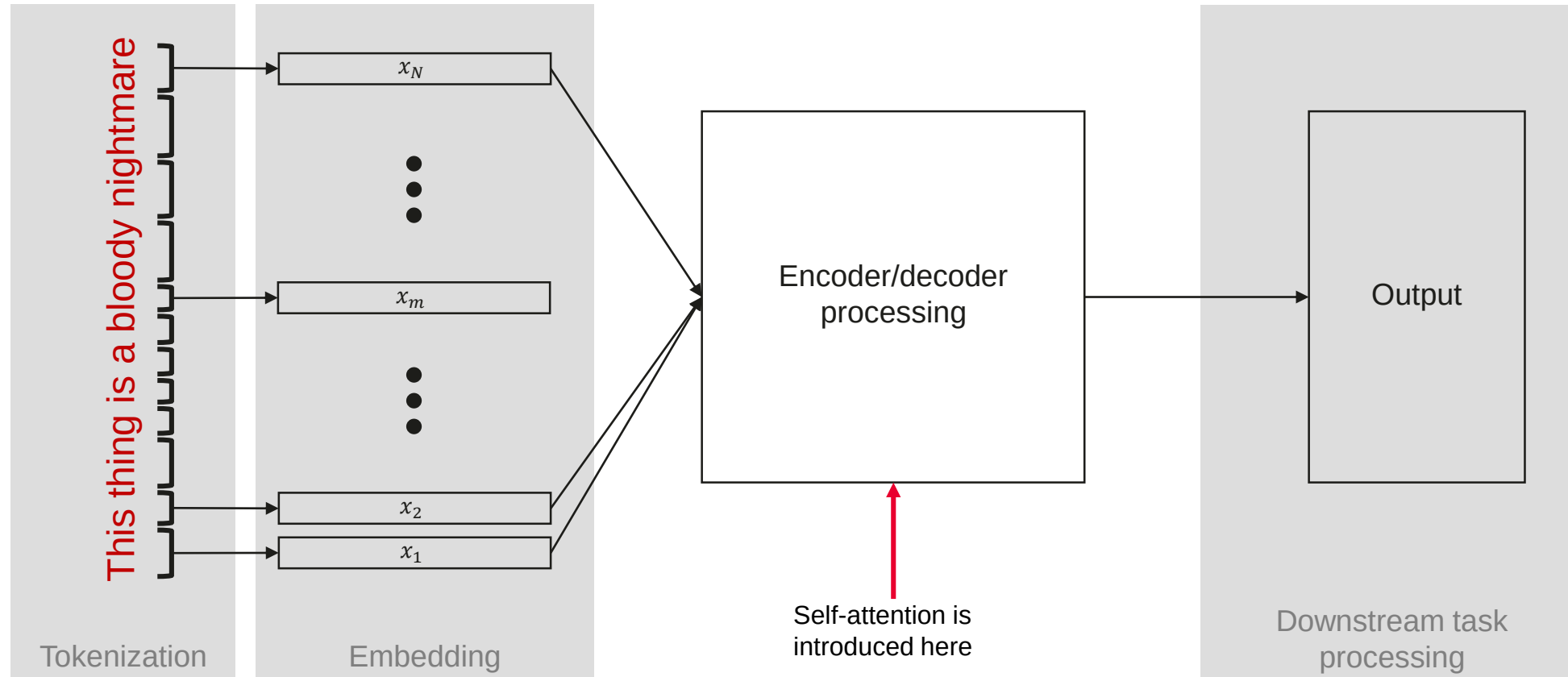


Attention mechanism background



NLP model architecture

N – context length (the number of input tokens)
 D – token embedding dimension
 $x_i \in \mathbb{R}^D$ – embedding of the i -th token
 $X \in \mathbb{R}^{D \times N}$ – embedding matrix



Dot-product self-attention

N – context length (the number of input tokens)
 D – token embedding dimension
 D_q – key/query/value dimension
 $x_i \in \mathbb{R}^D$ – embedding of the i -th token
 $X \in \mathbb{R}^{D \times N}$ – embedding matrix
 $\beta_* \in \mathbb{R}^{D_q}$, $\Omega_* \in \mathbb{R}^{D_q \times D}$ – key/query/value linear layer weight shared across token embeddings
 $K, V, Q \in \mathbb{R}^{D_q \times N}$ – key/query/value matrices

Matrix form

$$\begin{aligned}
 V &= \beta_v 1^T + \Omega_v X \\
 K &= \beta_k 1^T + \Omega_k X \\
 Q &= \beta_q 1^T + \Omega_q X \\
 Sa(X) &= V \cdot \text{Softmax}(K^T Q)
 \end{aligned}$$

Keys

$$k_m = \beta_k + \Omega_k x_m$$

Information retrieval terminology:
 “key” is something associated with the object to retrieve
 (e.g., video title)

$k_1, \dots, k_N$

Dot-product self-attention

$$a(x_m, x_n) = \text{softmax}_m(k_1^T q_n, \dots, k_N^T q_n) = \frac{\exp(k_m^T q_n)}{\sum_{l=1}^N \exp(k_l^T q_n)}$$

Attention $a(x_m, x_n)$ measures the attention paid by x_n to x_m and has probability-like properties:

- 1) $a(\cdot, x_n) \geq 0$
- 2) $\sum_m a(x_m, x_n) = 1$

Dot product acts as a similarity measure between a key and a query

$a(x_1, x_n), \dots, a(x_N, x_n)$

The n -th output of SA

$$sa_n(x_1, \dots, x_N) = \sum_{m=1}^N a(x_m, x_n) v_m$$

Softly selecting the value with the highest attention paid by x_n

$v_1, \dots, v_N$

Queries

$$q_n = \beta_q + \Omega_q x_n$$

Information retrieval terminology:
 “query” is something used to search for the object
 (e.g., text in the search bar)

$q_1, \dots, q_N$

Values

$$v_m = \beta_v + \Omega_v x_m$$

Information retrieval terminology:
 “value” is the object we want to retrieve (e.g., video)

Attention extensions

1 Scaled dot-product self-attention

[Vaswani et al. Attention is all you need \(2017\)](#)

Dot products in the attention may have large magnitudes which results in little sensitivity of the small changes of the inputs to the outputs in the softmax function. As such, the gradients will be very small. To resolve the problem, we scale the dot products:

$$Sa(X) = V \cdot \text{Softmax}\left(\frac{K^T Q}{\sqrt{D_q}}\right)$$

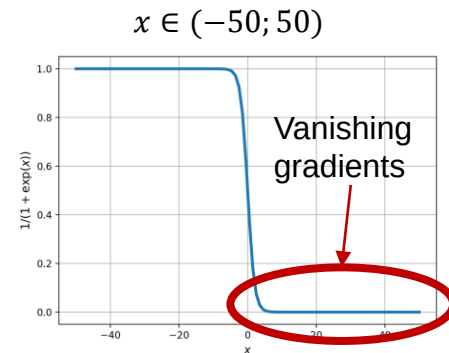
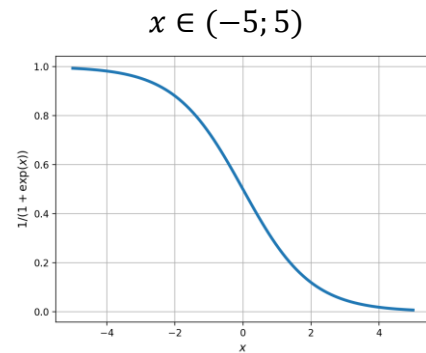
The square root appears due to a simple assumption:

$$k_i \sim n(0, 1)$$

$$q_i \sim n(0, 1)$$

$$\text{Var}[k^T q] = \text{Var}\left[\sum_{i=1}^{D_q} k_i q_i\right] = D_q \Rightarrow \text{Var}\left[\frac{k^T q}{\sqrt{D_q}}\right] = 1$$

Rule of thumb: $\text{Softmax}(x)$ has “good” values and gradients if $x \in (-5; 5)^N$

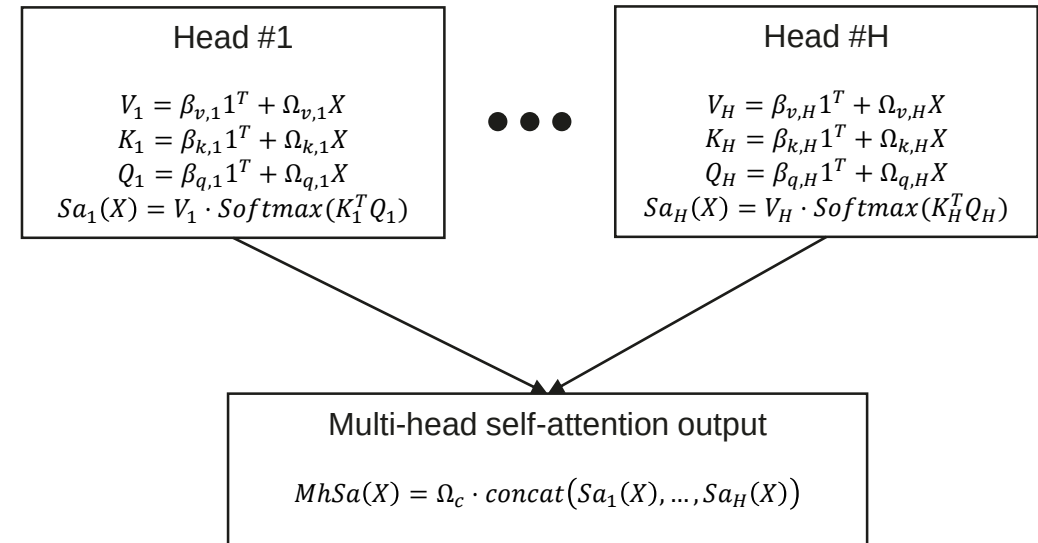


2 Multi-head attention (MHA)

[Vaswani et al. Attention is all you need \(2017\)](#)

Used by: Llama 2, T5

Instead of using one big attention head, multiple heads are usually applied whose keys, values and queries have dimensionality D/H . It seems to be necessary to use multiple heads for the self-attention mechanism to work well. One of the possible reasons is that they make the self-attention network more robust to bad initializations

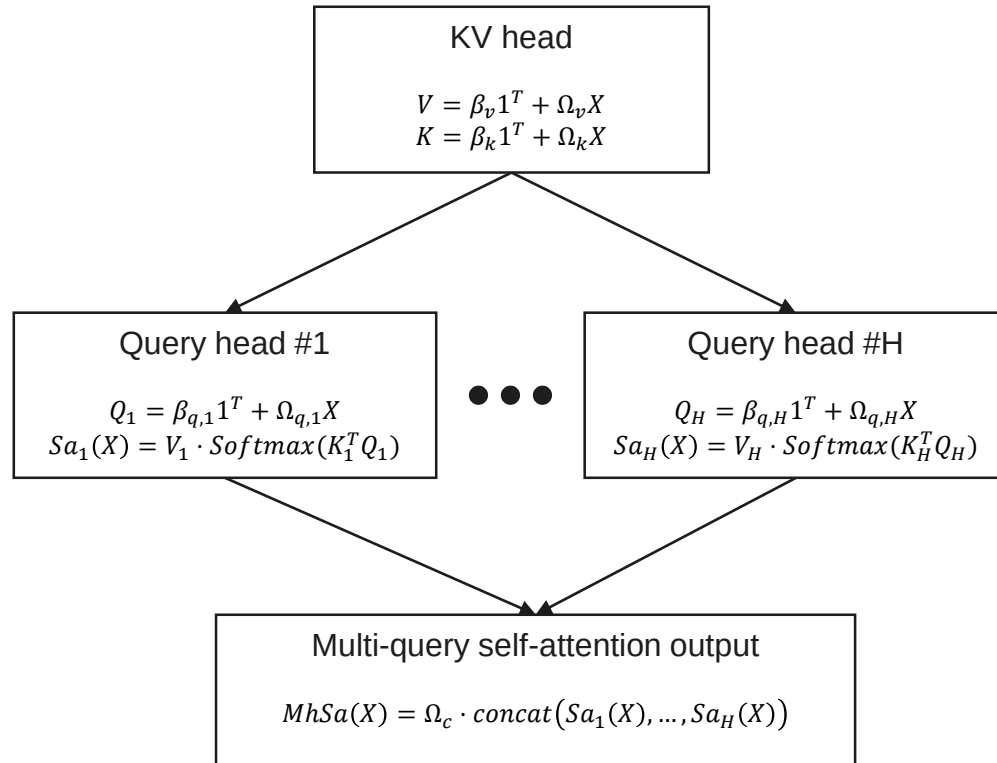


Attention extensions

3 Multi-query attention (MQA)

[Shazeer. Fast Transformer Decoding: One Write-Head is All You Need \(2019\)](#)
Used by: PaLM

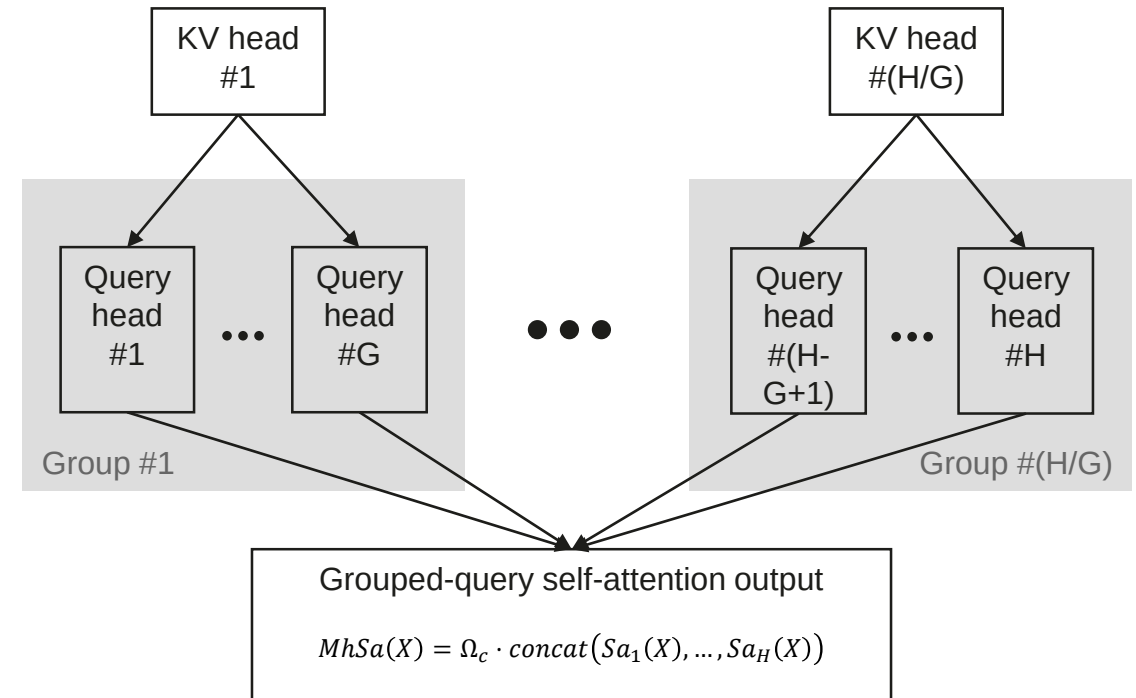
Multi-query attention is identical to multi-head attention except for keys and values being shared between heads. This results in the memory-bandwidth cost reduction which leads to inference acceleration. Shazeer (2019) showed that the accuracy of MQA has a relatively small decrease compared to MHA.



4 Grouped-query attention (GQA)

[Ainslie et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints \(2023\)](#)
Used by: Mistral

Grouped-query attention is merely MQA where query heads are grouped and there is a dedicated KV head per each group. This improves the accuracy while yielding almost the same inference acceleration as MQA.



Transformer layers and Models



Positional encoding

1 Absolute positional encoding

Vaswani et al. *Attention is all you need* (2017)
Used by: GPT-2

Since the dot product does not preserve the order of tokens, self-attention misses information about the sequence structure which is crucial for language and time series. This problem is resolved by adding a positional encoding. As the name suggests, it encodes the position of a token in a sentence.

Absolute positional embedding by Vaswani et al. (2017) simply adds a positional embedding matrix $P \in \mathbb{R}^{D \times N}$ to X so that $X + P$ is passed to the attention layer instead of X . Matrix P is fixed and made of harmonics:

$$P_{i,2j} = \sin\left(\frac{i}{10000^{\frac{2j}{D}}}\right)$$
$$P_{i,2j+1} = \cos\left(\frac{i}{10000^{\frac{2j}{D}}}\right)$$

The intuition behind this encoding comes from the binary representation of position numbers:

0	1	2	3	4	5	6	7
0	1	0	1	0	1	0	1
0	0	1	1	0	0	1	1
0	0	0	0	1	1	1	1

→ High frequency

→ Low frequency

2 Relative positional encoding

Shaw et al. *Self-Attention with Relative Position Representations* (2018)

Instead of encoding the exact position in a sequence, it may be more useful to encode $n - m$, i.e. the distance in positions between x_n and x_m . Shaw et al. (2018) suggested using trainable p_r^k and p_r^v , where $r = \text{clip}(m - n, r_{\min}, r_{\max})$, which is added to x_m for keys and values only while removing the biases:

$$k_m = \Omega_k(x_m + p_r^k)$$

$$v_m = \Omega_v(x_m + p_r^v)$$

Clipping is used because the relative position becomes invalid beyond a certain distance

Dependence on the clipping distance

$r_{\max}$ (k in the table)

k	EN-DE BLEU
0	12.5
1	25.5
2	25.8
4	25.9
16	25.8
64	25.9
256	25.8

Comparison between absolute and relative positional encoding based on translation tasks

Model	Position Information	EN-DE BLEU	EN-FR BLEU
Transformer (base)	Absolute Position Representations	26.5	38.2
Transformer (base)	Relative Position Representations	26.8	38.7
Transformer (big)	Absolute Position Representations	27.9	41.2
Transformer (big)	Relative Position Representations	29.2	41.5

Positional encoding

3 Rotary position embedding (RoPE)

Su et al. RoFormer: Enhanced Transformer with Rotary Position Embedding (2021)
Used by: Llama 2, Mistral

RoPE aims at incorporating relative position information to the inner product $k_m^T q_n$:

$$\langle k_m, q_n \rangle = g(x_m, x_n, n - m)$$

where $g(\cdot, \cdot, \cdot)$ is some function to be defined. The solution for g is simply

$$k_m = \Omega_k x_m \exp(im\theta)$$

$$q_n = \Omega_q x_n \exp(in\theta)$$

$$g(x_m, x_n, n - m) = \text{Re}[(\Omega_k x_m)^* (\Omega_q x_n) \exp(i(n - m)\theta)]$$

Here $\theta \in \mathbb{R}$ is a prescribed constant.

In 2D, it is equivalent to rotating an affine-transformed token embedding by a multiple of the position index:

$$k_m = R_{\theta, m} \Omega_k x_m$$

$$q_n = R_{\theta, m} \Omega_q x_n$$

$$g(x_m, x_n, n - m) = x_n^T \Omega_q R_{\theta, m} \Omega_k x_m$$

where $R_{\theta, m}$ is the rotation matrix for angle $m\theta$.

For the D -dimensional embedding, we split x_m into 2D chunk and rotate each of the in the 2D fashion:

$$R_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

Following Vaswani, RoPE sets $\theta_i = 10000^{-2i/D}$

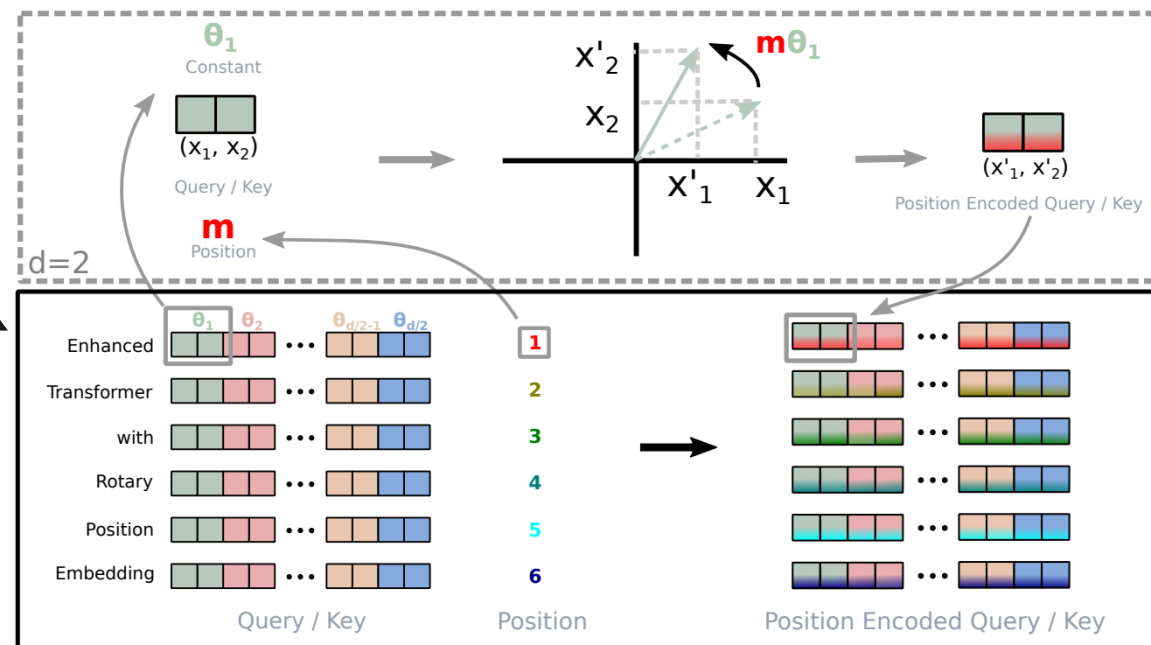
4 NTK-aware RoPE

Roziere et al. Code Llama: Open Foundation Models for Code (2024)

Used by: CodeLlama (based on Llama 2)

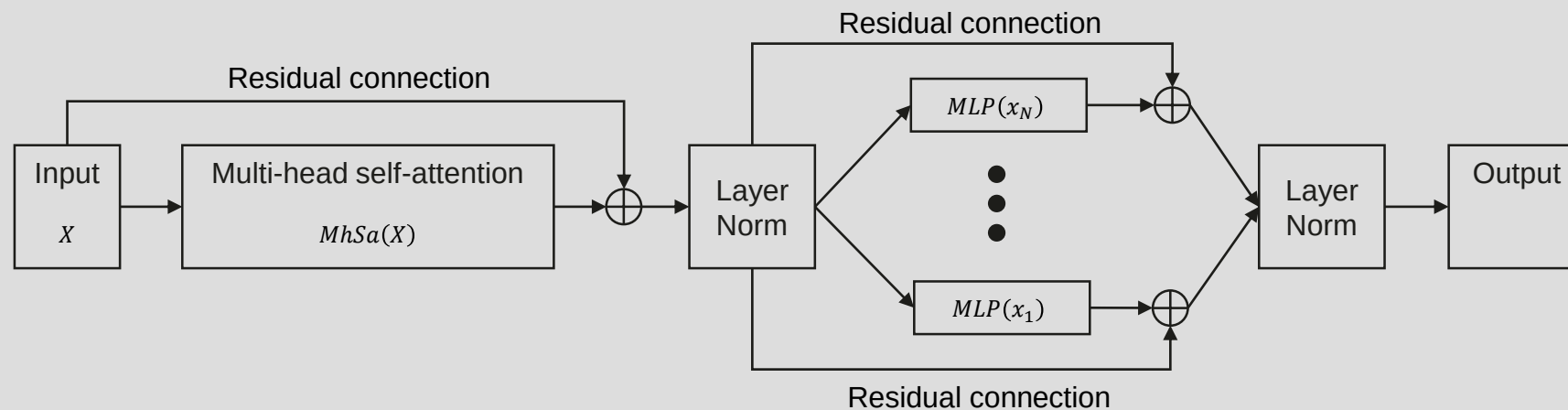
5 YaRN (ROPE extension)

Peng et al. YaRN: Efficient Context Window Extension of Large Language Models (2023)



Transformer model

Transformer layer



Matrix form

$$\begin{aligned} X &\leftarrow X + \text{MhSa}(X) \\ X &\leftarrow \text{LayerNorm}(X) \\ x_n &\leftarrow x_n + \text{MLP}(x_n), n = 1, \dots, N \\ X &\leftarrow \text{LayerNorm}(X) \end{aligned}$$

Transformer model

Vaswani et al. Attention is all you need (2017)

Transformer model is simply a series of K transformer layers. There are three types of transformer models:

- Encoders (text representation supporting a variety of tasks)
- Decoders (next token prediction)
- Encoder-decoders (seq-to-seq tasks)

Layer normalization (LayerNorm)

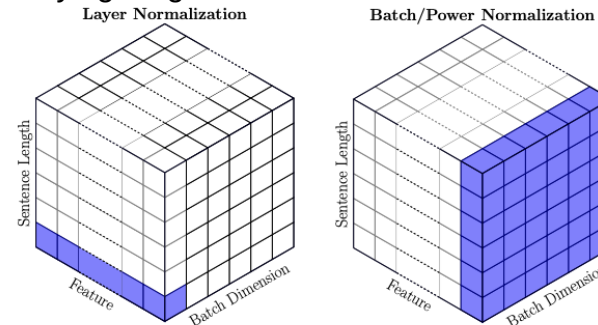
Ba et. al. Layer normalization (2016)

Each batch element and each token are normalized separately using the statistics gathered across the non-batch dimensions.

$$Y \leftarrow \gamma \odot \frac{Y - E[Y]}{\sqrt{\text{Var}[Y] + \epsilon}} + \beta$$

where $Y = X_{n,:}$, $E[.]$ and $\text{Var}[.]$ are computed with respect to non-batch/non-token dimensions and $\odot$ is the Hadamard product so γ and β are matrices having the same dimensionality as Y and are tuned during the training.

LayerNorm (left) is usually used in NLP while BatchNorm (right) is usually used in CV. This difference comes from the importance of gathering statistics for words separately and the varying length of sentences

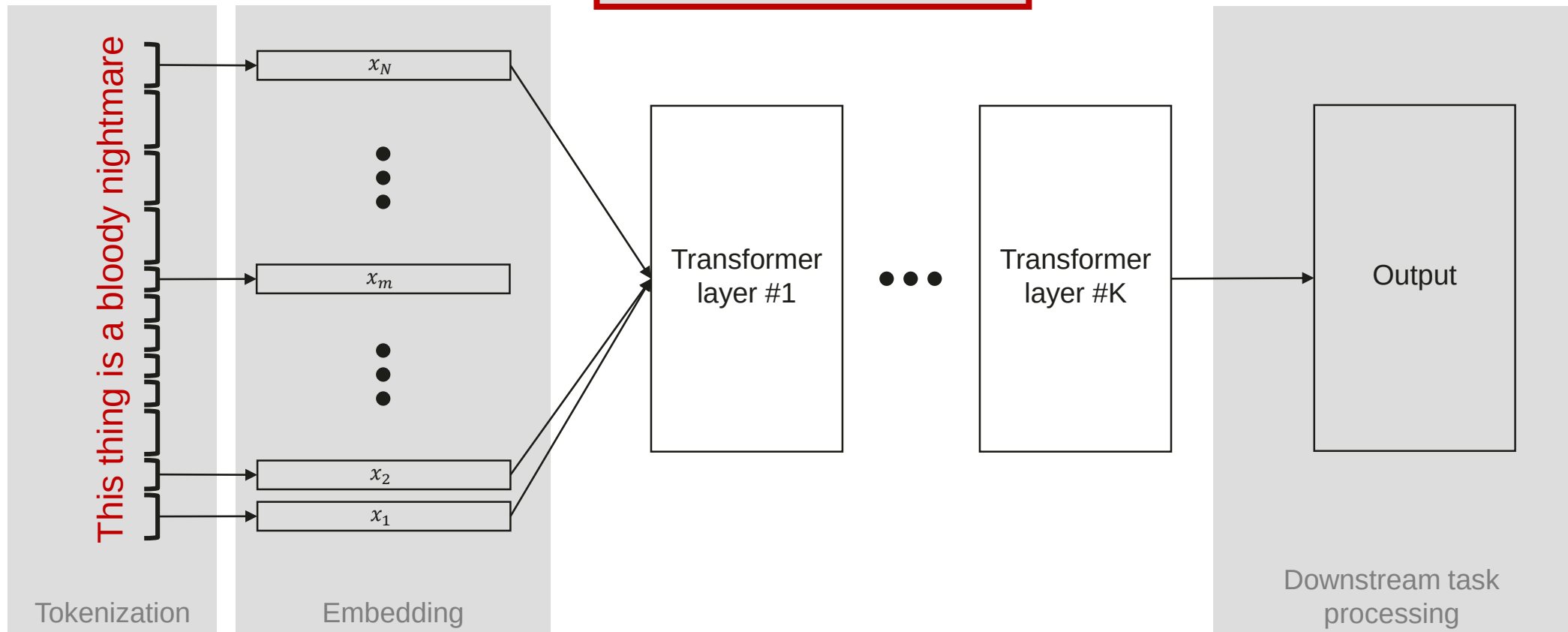


Abuse of language: Layer Normalization used in transformers is formally called Instance Normalization while the true Layer Normalization would also aggregate statistics over token

Encoder model: a representational transformer

Key idea

An encoder transforms the text embeddings into a representation that can support a variety of tasks



Encoder model: a representational transformer

1 Example: BERT

[Devlin et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding \(2018\)](#)

Vocabulary: $M = 30\text{k}$ tokens

Maximum context length: $N = 512$ tokens

Token dimension: $D = 1024$

K, Q and V dimension: $D_q, D_k, D_v = 64$

Number of heads: $H = 16$

Number of transformer layers: $K = 24$

Hidden MLP layer dimension: $D_{MLP} = 4096$

Total number of parameters: 340M

2 Pre-training

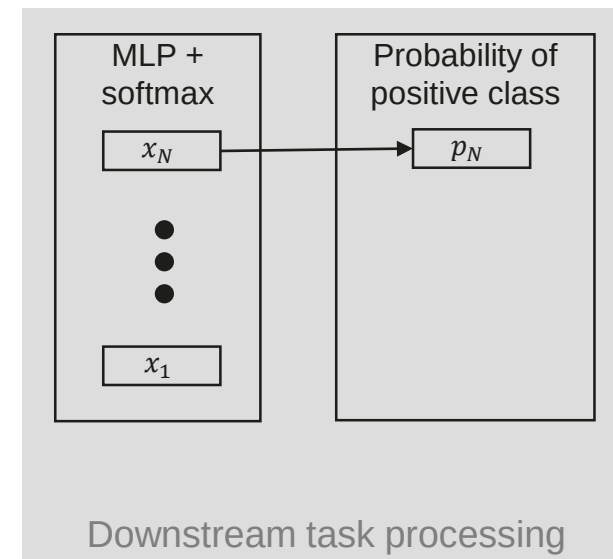
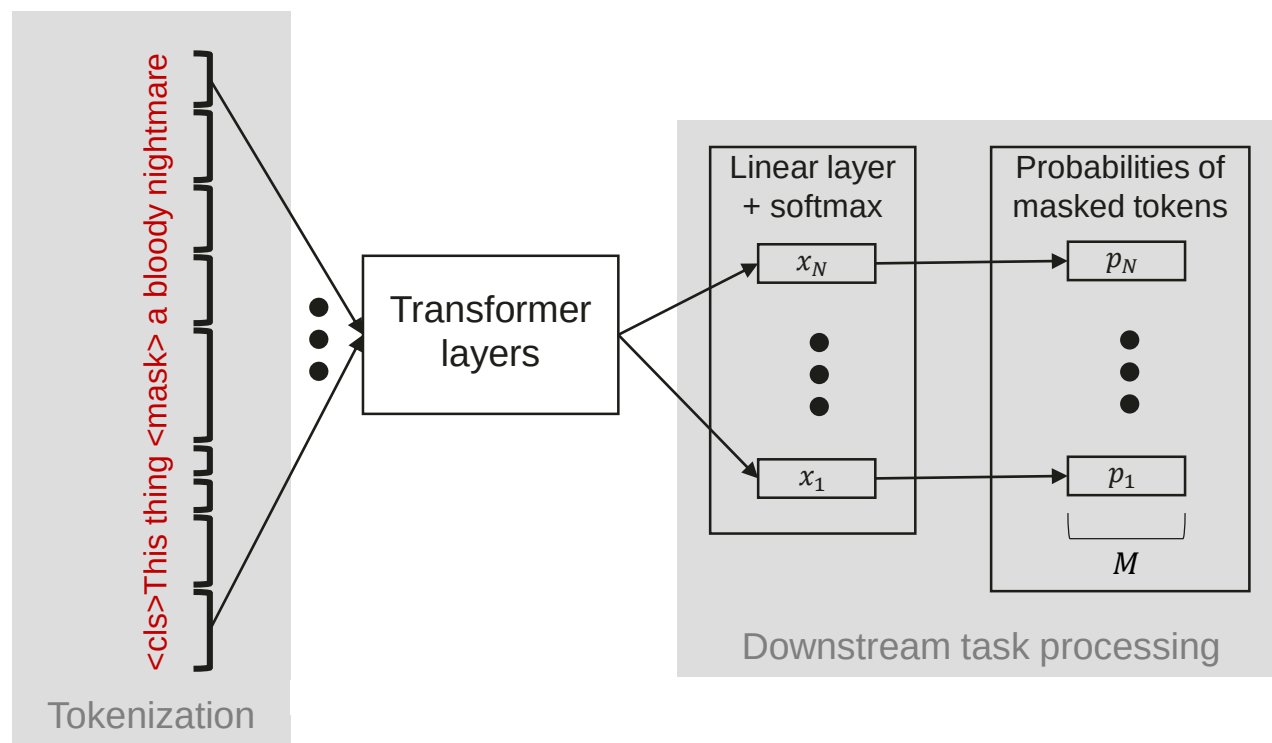
The network is trained using self-supervision: in this case, predicting missing words marked by `<mask>` from sentences from a large text corpus. This forces the model to use both the left and right context to predict the missing token. The training dataset is prepared by randomly replacing some words with `<mask>` tokens.

3 Fine-tuning

After pre-training, the mask-processing head is removed and replaced with a specialized head appended onto the network of transformer layers to convert the output vectors to the desired output format

- **Text classification:** the vector associated with the `<cls>` token is mapped to a probability
- **Word classification:** similar to the pre-training head, but we need to predict the word class

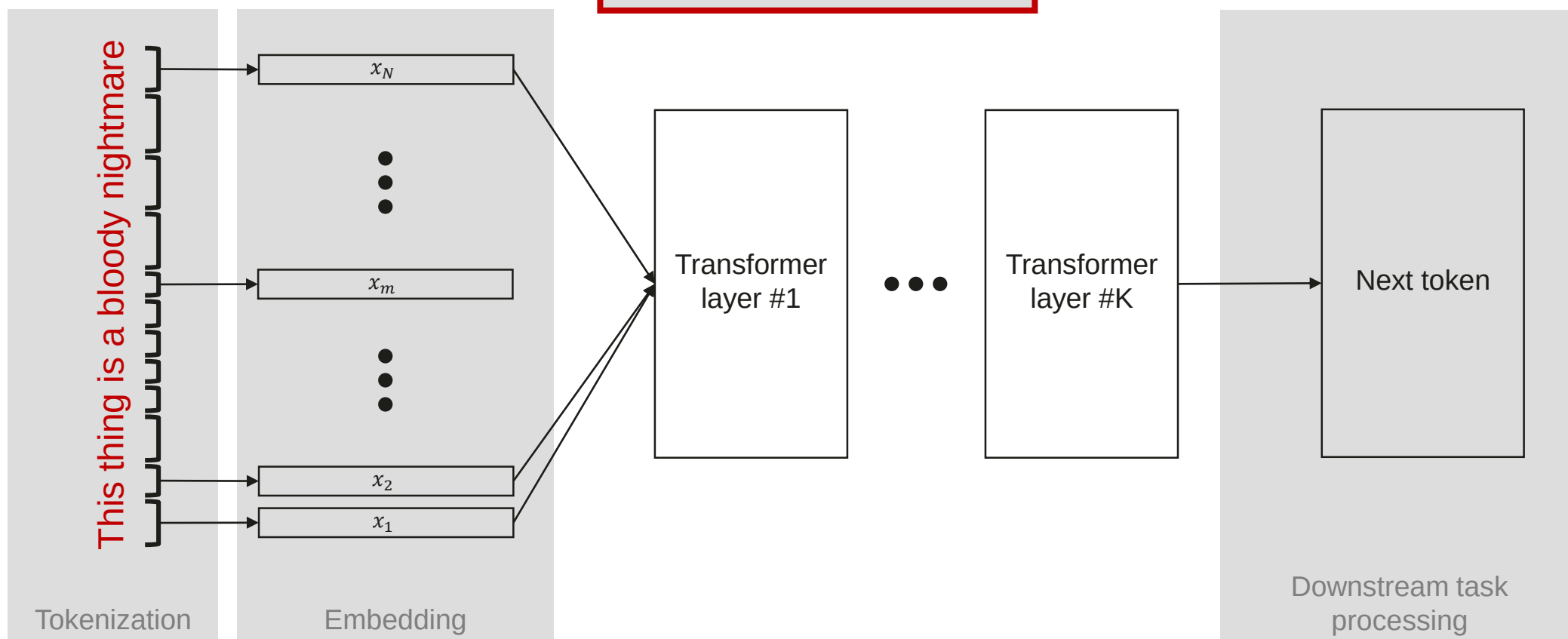
The model parameters are then fine-tuned according to the downstream task



Decoder model: a generative transformer

Key idea

A decoder is similar to an encoder. The key difference is that a decoder does only one job: generating the next token in a sentence



Decoder model: a generative transformer

1 Example: GPT3

Brown et al. Language Models are Few-Shot Learners (2020)

Vocabulary: $M = XX$ tokens

Maximum context length: $N = 2048$ tokens

Token dimension: $D = 12288$

K, Q and V dimension: $D_q, D_k, D_v = 128$

Number of heads: $H = 96$

Number of transformer layers: $K = 96$

Hidden MLP layer dimension: $D_{MLP} = XX$

Total number of parameters: 175B

2 Pre-training

A decoder model follows the **autoregressive language model** factoring the joint probability into conditionals:

$$P(x_1, \dots, x_L) = P(x_1) \prod_{i=2}^L P(x_i | x_1, \dots, x_{i-1})$$

Pre-training focuses on maximizing the log probability of the input text. Note that this is equivalent to maximizing a sum of log probabilities of next-token predictions. Multiclass cross-entropy is used as a loss function

3 Masked self-attention

An autoregressive model implies that we must not look beyond the last token during the next-token prediction. However, the context contains all the tokens and this assumption will be violated while computing self-attention. To eliminate this problem, masked self-attention is used:

$$\text{MaskedSa}(X) = V \cdot \text{Softmax}(K^T Q + M)$$

where $M \in \mathbb{R}^{N \times N}$ is a mask whose strictly upper triangular part is filled by negative infinity:

$$M = \begin{bmatrix} 0 & -\infty & \dots & -\infty \\ 0 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & -\infty \\ 0 & \dots & 0 & 0 \end{bmatrix}$$

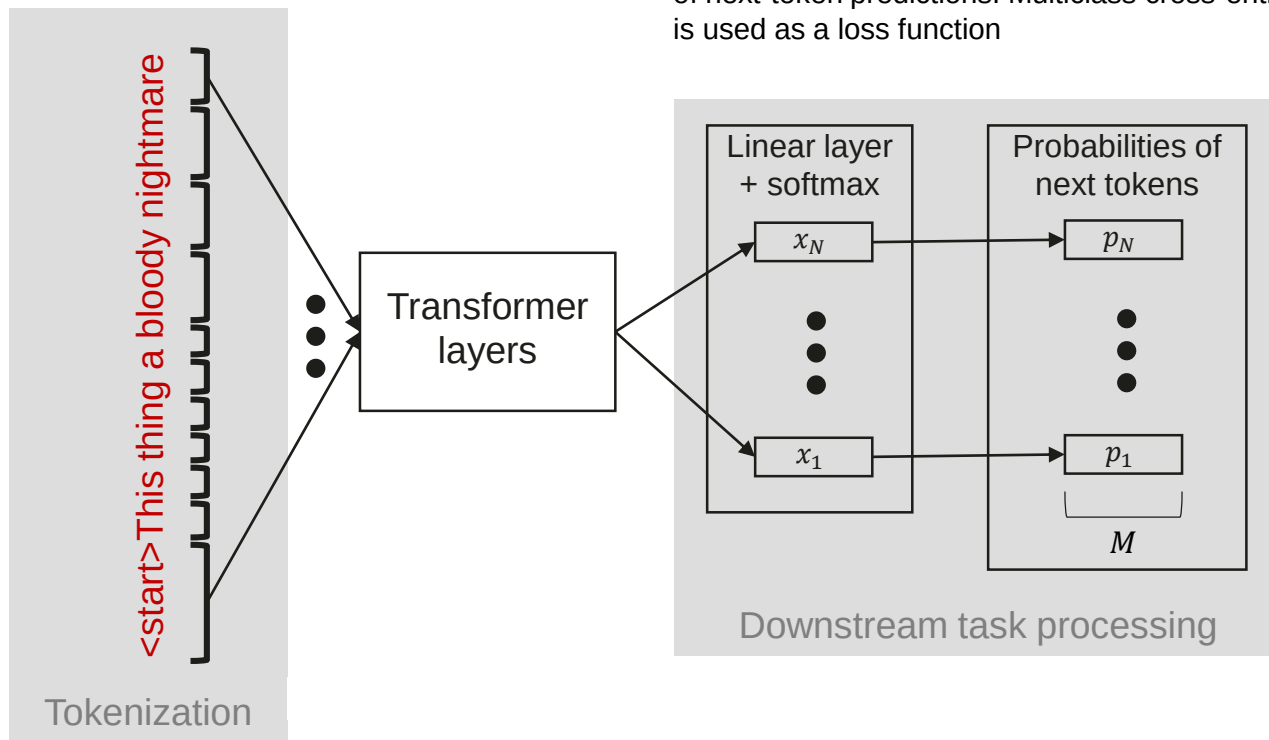
This is equivalent to setting $a(x_m, x_n) = 0$ for $m > n$

4 Generating text from a decoder

We can start from an input sequence of text or simply the <start> token indicating the beginning of the sequence. Different decoding strategies can be used to select the next token prediction:

- **Greedy decoding:** pick the most probable one
- **Beam search:** keep track of multiple possible sentence completions and find the overall most probable one
- **Top-k sampling:** randomly pick one of the top-K most probable ones

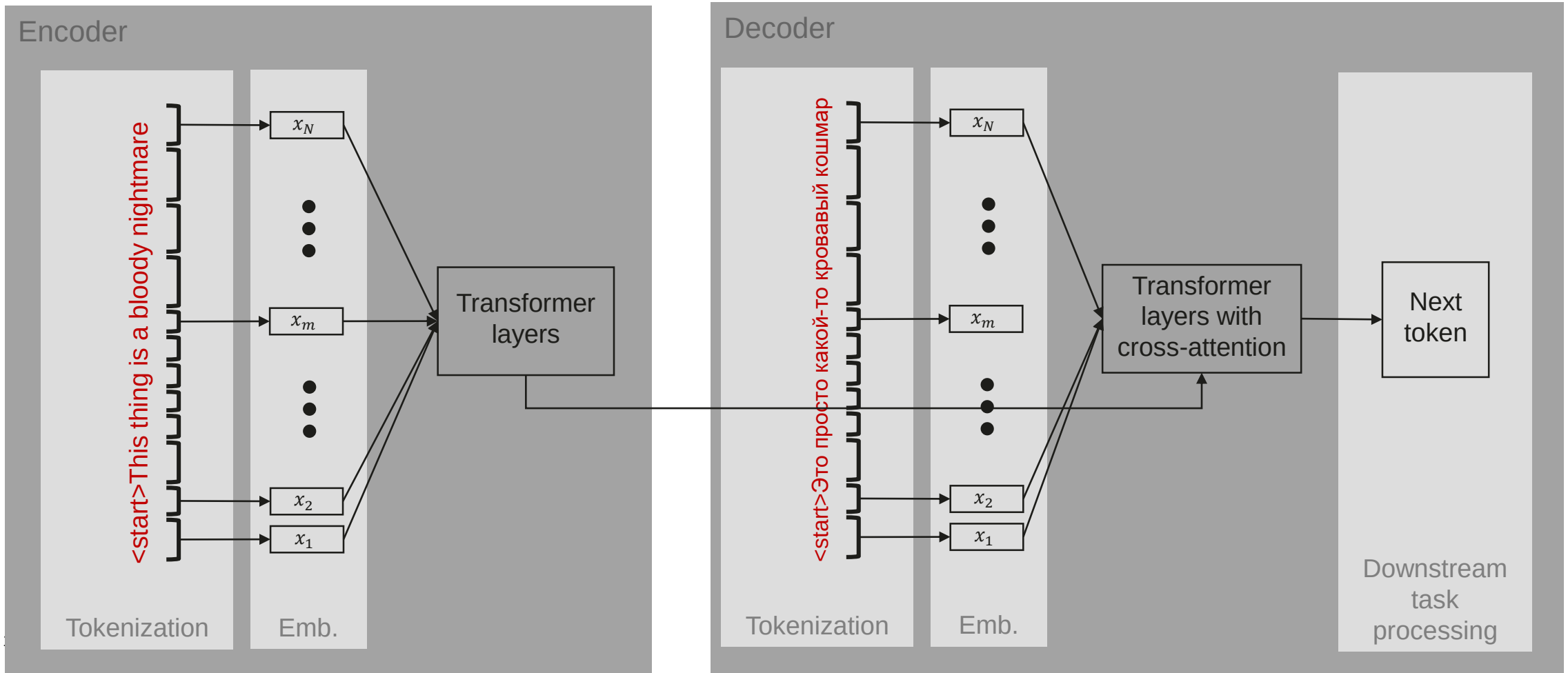
Note that the autoregressive nature of transformers allows for caching strategies optimizing the inference time: all the $a(x_m, x_n)$ except for the last row will be the same when we add the next token to the input context and try to predict the token after the next token



Encoder-decoder model: a seq2seq transformer

Key idea

An encoder-decoder model takes the encoder to build a good sentence representation and the decoder to generate the output sentence



Encoder-decoder model: a seq2seq transformer

1 Cross attention

Cross-attention takes X for K and V from the encoder and X for Q from the decoder:

$$V = \beta_v 1^T + \Omega_v X_{enc}$$

$$K = \beta_k 1^T + \Omega_k X_{enc}$$

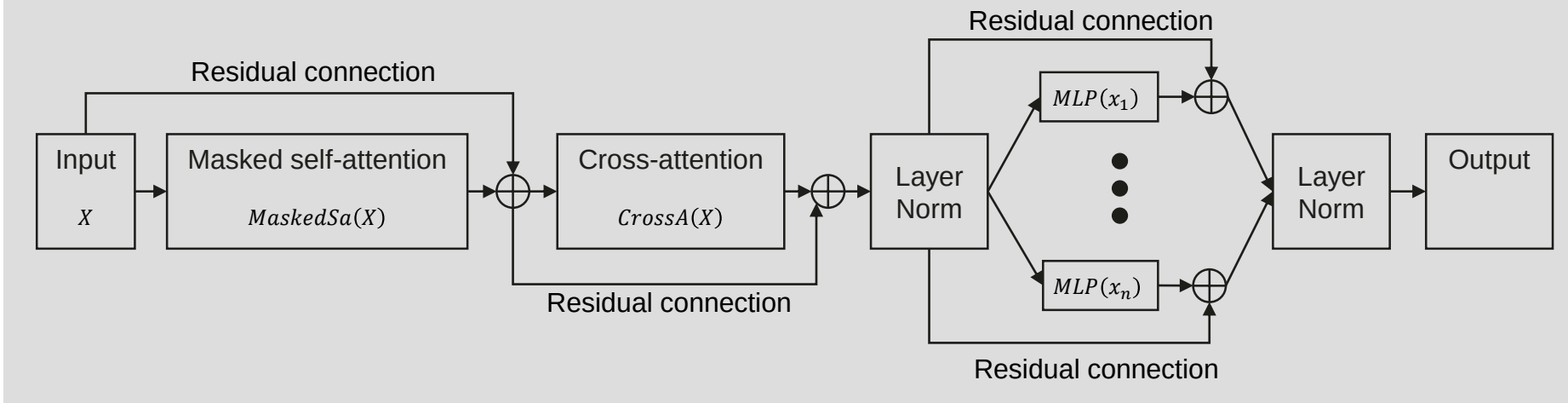
$$Q = \beta_q 1^T + \Omega_q X_{dec}$$

$$CrossA(X) = V \cdot Softmax(K^T Q)$$

2 Transformer layers in the decoder

These transformer layers only differ in the fact that right after a masked self-attention layer there is a cross-attention layer

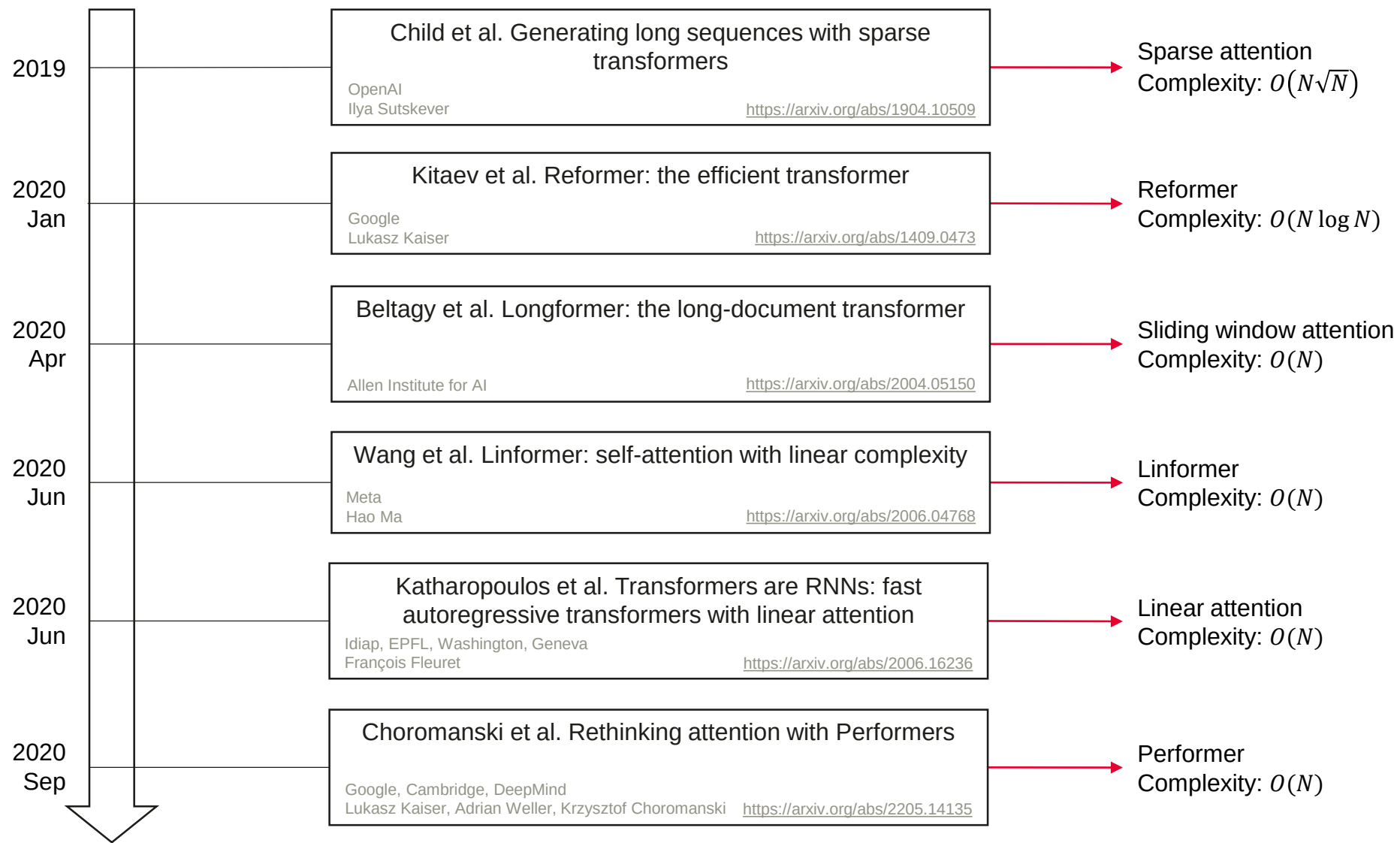
Transformer layer



Quadratic complexity



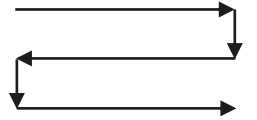
Reducing the quadratic complexity of attention



Sparse attention

Child et al. Generating long sequences with sparse transformer (2019)
Used in: GPT-3

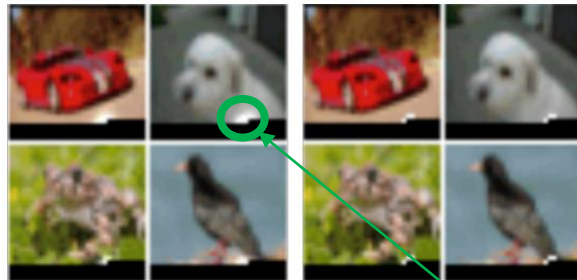
Image as a sequence



1 Sparsity of attention patterns

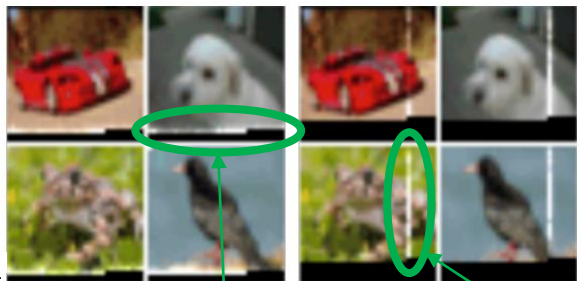
Using images as an example, it was shown that attention matrices are characterized by sparsity patterns. Below white means high attention and black means the autoregressive mask for next pixel prediction

Spatial locality
(layers 1 to 18)



Last k pixels +
strides

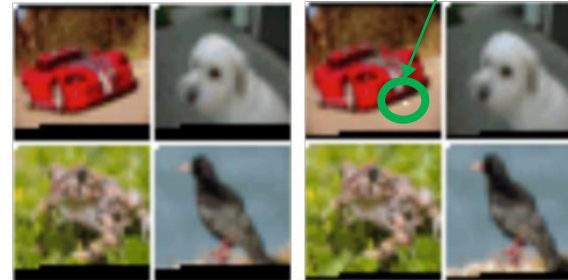
Row / column locality
(layers 19 to 20)



Row locality

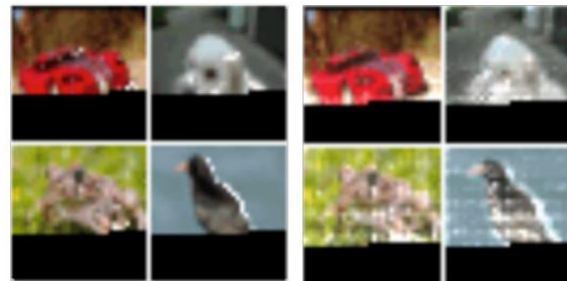
Column locality

High sparsity with positions activated for
specific inputs only
(typical layers 64 to 128)



Specific position

Global patterns
(some layers)



2 Factorized self-attention

These patterns can partly be reflected in self-attention by parameterizing a connectivity pattern.

Suppose we wish to compute attention x_n pays to other tokens and let S_n be the index set for all the tokens to be checked.

- Full self-attention for autoregressive models:

$$S_n = \{m: m \leq n\}$$

- Factorized self-attention splits self-attention into p separate attention heads and then for each attention head define $S_n^{(p)} \subset S_n$ so that two conditions are satisfied:

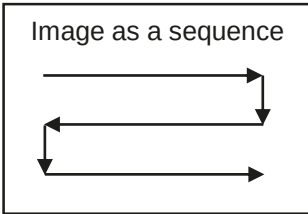
1. $|S_n^{(p)}| \propto n^{1/p}$

2. For every $m \leq n$, $S_n^{(p)}$ are defined so that n can attend m through a path of locations with maximum length $p + 1$

This gives $O(N \times N^{1/p})$ complexity

Sparse attention

Child et al. Generating long sequences with sparse transformer (2019)
Used in: GPT-3



3 Two-dimensional factorized attention ($p = 2$)

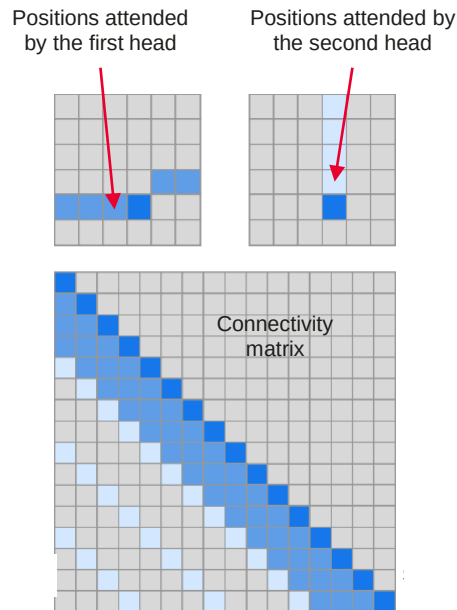
Strided attention:

The first head attends the previous l positions, the second head attends every l -th position where $l \approx \sqrt{N}$ is the stride:

$$S_n^{(1)} = \{n - l, n - l + 1, \dots, n\}$$

$$S_n^{(2)} = \{m: (n - m) \bmod l = 0\}$$

It works well for data with periodic structure (images, music)



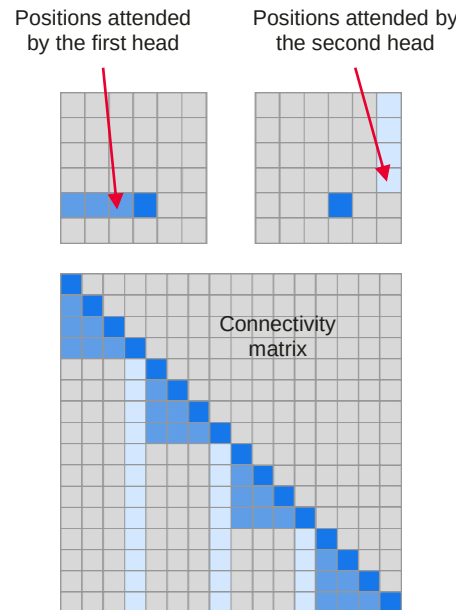
Fixed attention:

The first head attends several previous positions starting from a fixed position to the current one. The second head attends every l -th position with shifts:

$$S_n^{(1)} = \left\{ m: \left\lfloor \frac{m}{l} \right\rfloor = \left\lfloor \frac{n}{l} \right\rfloor \right\}$$

$$S_n^{(2)} = \{m: m \bmod l \in \{l - c, l - c + 1, \dots, l\}\}$$

It works well for text data



4 Results: accuracy

Sparse attention enables long-context processing $\Rightarrow$ long-term dependencies $\Rightarrow$ increased accuracy

Model	Bits per byte
CIFAR-10	
PixelCNN (Oord et al., 2016)	3.03
PixelCNN++ (Salimans et al., 2017)	2.92
Image Transformer (Parmar et al., 2018)	2.90
PixelSNAIL (Chen et al., 2017)	2.85
Sparse Transformer 59M (strided)	2.80
Enwik8	
Deeper Self-Attention (Al-Rfou et al., 2018)	1.06
Transformer-XL 88M (Dai et al., 2018)	1.03
Transformer-XL 277M (Dai et al., 2018)	0.99
Sparse Transformer 95M (fixed)	0.99
ImageNet 64x64	
PixelCNN (Oord et al., 2016)	3.57
Parallel Multiscale (Reed et al., 2017)	3.7
Glow (Kingma & Dhariwal, 2018)	3.81
SPN 150M (Menick & Kalchbrenner, 2018)	3.52
Sparse Transformer 152M (strided)	3.44

Just re-scaled NLL

5 Results: inference acceleration

Less attention weights to be computed $\Rightarrow$ inference acceleration

Model	Bits per byte	Time/Iter
Enwik8 (12,288 context)		
Dense Attention	1.00	1.31
Sparse Transformer (Fixed)	0.99	0.55
Sparse Transformer (Strided)	1.13	0.35
CIFAR-10 (3,072 context)		
Dense Attention	2.82	0.54
Sparse Transformer (Fixed)	2.85	0.47
Sparse Transformer (Strided)	2.80	0.38

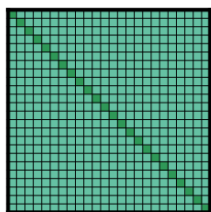
Sliding window attention (Longformer)

Beltagy et al. Longformer: the long-document transformer (2020)

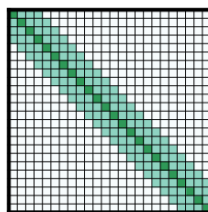
Used in: Mistral

1 More attention patterns

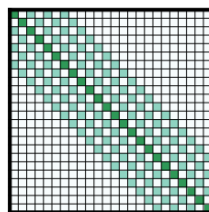
The idea and the motivation are similar to Child et al. (2019), but more patterns are explored. Sliding window is one of them, and dilated sliding window and global+sliding window are two other options



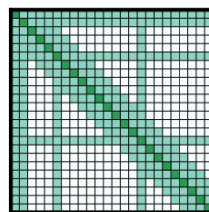
(a) Full n^2 attention



(b) Sliding window attention



(c) Dilated sliding window



(d) Global+sliding window

Sliding window attention: just a fixed-size window w surrounding each token. It is assumed that multiple stacked layers of such attentions result in a large receptive field

Dilated sliding window: the window has gaps of size dilation d ; same idea as in dilated CNNs

Global attention: specific solution for encoder models aimed at representation tasks which may include global context (e.g., text classification with `<cls>` token in BERT). Basically, the whole idea is to enable global attention at pre-selected input locations (e.g., `<cls>` token)

Mistral case: sliding window attention with $w = 4096$

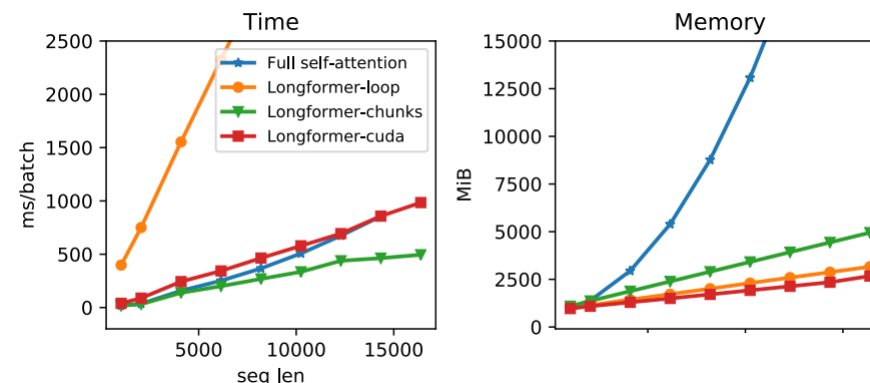
2 Results: accuracy

When pre-trained, Longformer consistently outperformed RoBERTa on long document tasks and set new SOTA results at that time on WikiHop and TriviaQA

Model	QA			Coref.	Classification	
	WikiHop	TriviaQA	HotpotQA	OntoNotes	IMDB	Hyperpartisan
RoBERTa-base	72.4	74.3	63.5	78.4	95.3	87.4
Longformer-base	75.0	75.2	64.4	78.6	95.7	94.8

3 Results: inference acceleration

Theoretically and empirically, both time- and space-complexity are shown to be linear



Other techniques

1 Locality-sensitive hashing attention (Reformer)

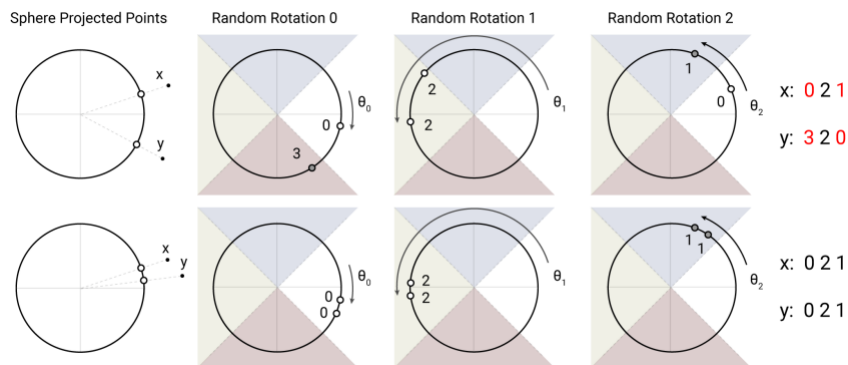
[Kitaev et al. Reformer: the efficient transformer \(2020\)](#)

Instead of computing attention exactly, we can approximate its value. This is motivated by the observation that softmax is dominated by the largest elements so that the contribution of other dot products can be neglected

To find the largest elements, we turn to the original formulation of self-attention via similarity metric and assume $Q = K$ (i.e., same linear layers are used to compute Q and K). Thus, for each query q_n we only need to find the closest keys $\Rightarrow$ nearest-neighbours problem

The problem of finding nearest neighbours quickly in high-dimensional spaces is solved by locality-sensitive hashing (i.e., hashing where nearby vectors get the same hash with high probability)

Random rotations are used to implement locality-sensitive hashing:



2 Linear attention (Linformer)

[Wang et al. Linformer: self-attention with linear complexity \(2020\)](#)

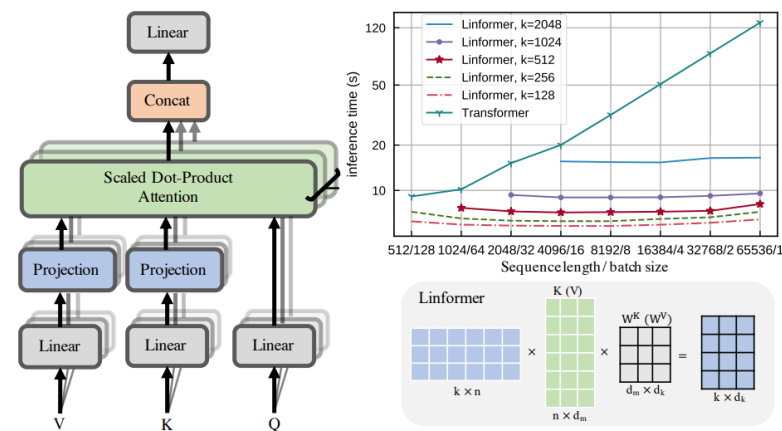
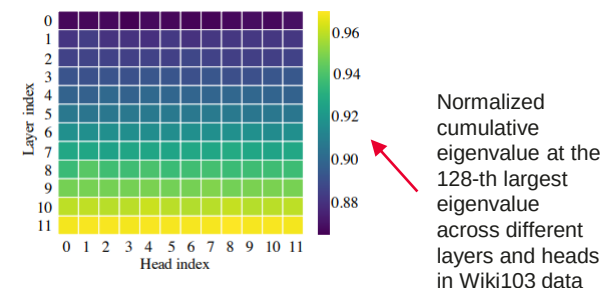
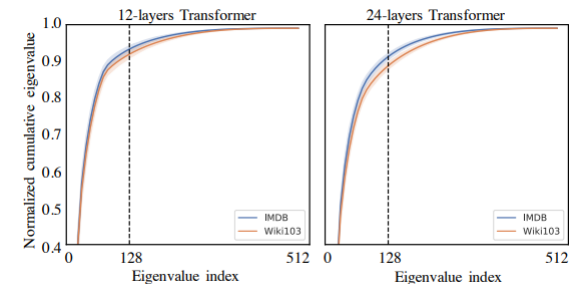
For the i -th head, the matrix induced by softmax has the following form:

$$P = \text{softmax} \left[\frac{QW_i^Q (KW_i^K)^T}{\sqrt{D_q}} \right]$$

where W_i^Q and W_i^K are just some trained matrices

This matrix appears to be low-rank. We could project it directly, but it is time-consuming. Instead, we can project keys and values onto matrices $E_i \in \mathbb{R}^{N \times k}$ and $F_i \in \mathbb{R}^{N \times k}$ where $k \ll N$:

$$Sa = \text{softmax} \left[\frac{QW_i^Q (E_i K W_i^K)^T}{\sqrt{D_q}} \right] F_i V W_i^V$$



Other techniques

3 Linear attention (Linear Transformer)

Katharopoulos et al. Transformers are RNNs: fast autoregressive transformers with linear attention (2020)

Self-attention we considered is actually a particular implementation of the self-attention idea which could be called softmax attention:

$$Sa(X) = V \cdot \text{Softmax}(K^T Q)$$

We can generalize it to an arbitrary similarity function akin to Nadaraya-Watson regression:

$$Sa_n(X) = \frac{\sum_{m=1}^N \text{sim}(q_n, k_m) v_m}{\sum_{m=1}^N \text{sim}(q_n, k_m)}$$

Here $\text{sim}(q, k) = \exp(q^T k / \sqrt{D_Q})$ would correspond to the softmax attention.

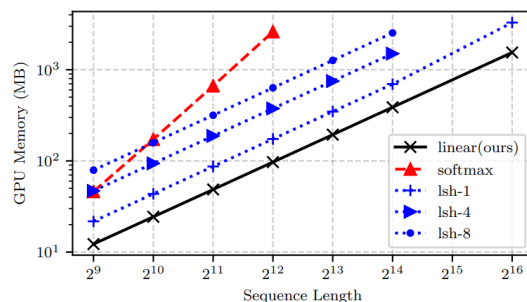
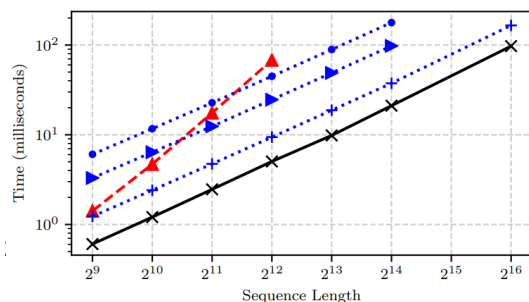
Now we can kernelize this attention, i.e. choose some kernel $k(x, y)$ which implies a feature map $\phi(x)$ and turns the similarity function into

$$Sa_n(X) = \frac{\sum_{m=1}^N \phi(q_n)^T \phi(k_m) v_m}{\sum_{m=1}^N \phi(q_n)^T \phi(k_m)}$$

which can be a bit simplified from the computational viewpoint:

$$Sa_n(X) = \frac{\phi(q_n)^T \sum_{m=1}^N \phi(k_m) v_m^T}{\phi(q_n)^T \sum_{m=1}^N \phi(k_m)}$$

Note that highlighted terms can be computed only once and then used for all the queries.

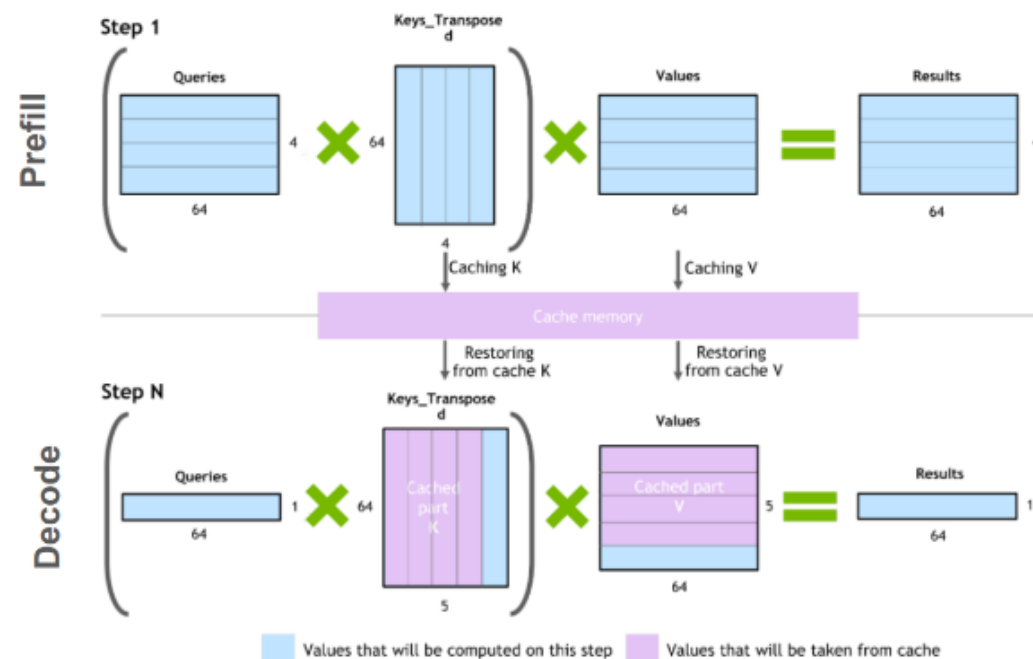


4 KV cache

Used in: the vast majority of LLMs

A very simple idea: in the next token prediction mode, avoid re-computing all the keys and values except for the very last one by storing them in the cache memory

$(Q * K^T) * V$ computation process with caching



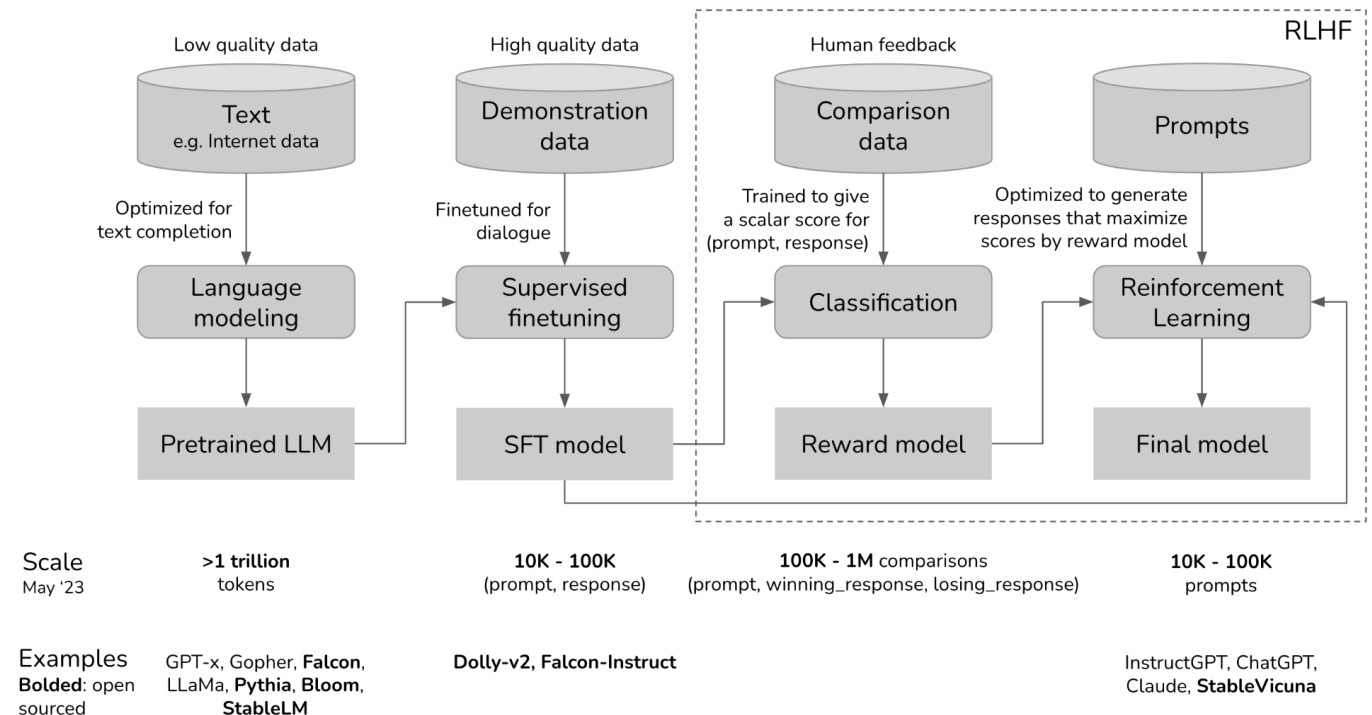
LLM training pipeline



LLM training pipeline

There are 2 (3?) stages of model training:

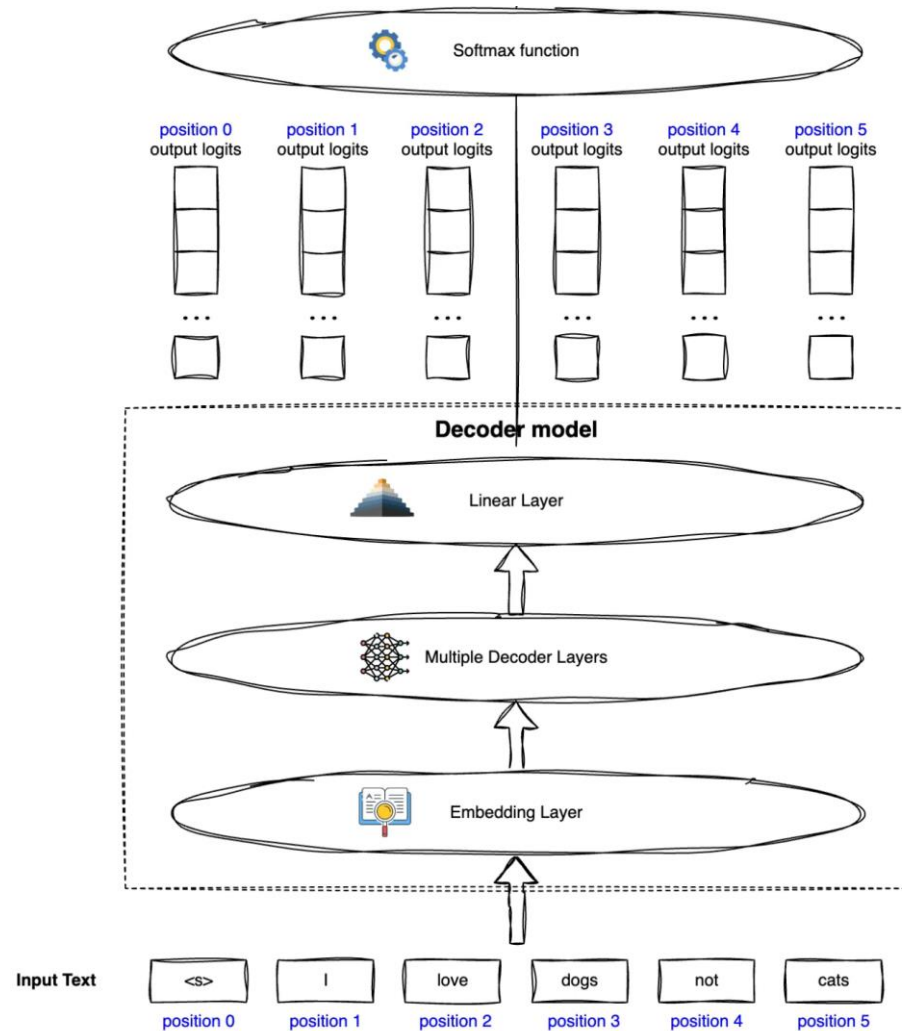
1. Pre-train. Train model to predict next token on a colossal data set (trillions/tens of trillion tokens)
2. Post-train / Alignment. In order for the LLM to be useful it needed to be fine-tuned to a specific task(s) with labeled data (SFT). After that, it can be additionally aligned “towards human goals, preferences and principles”



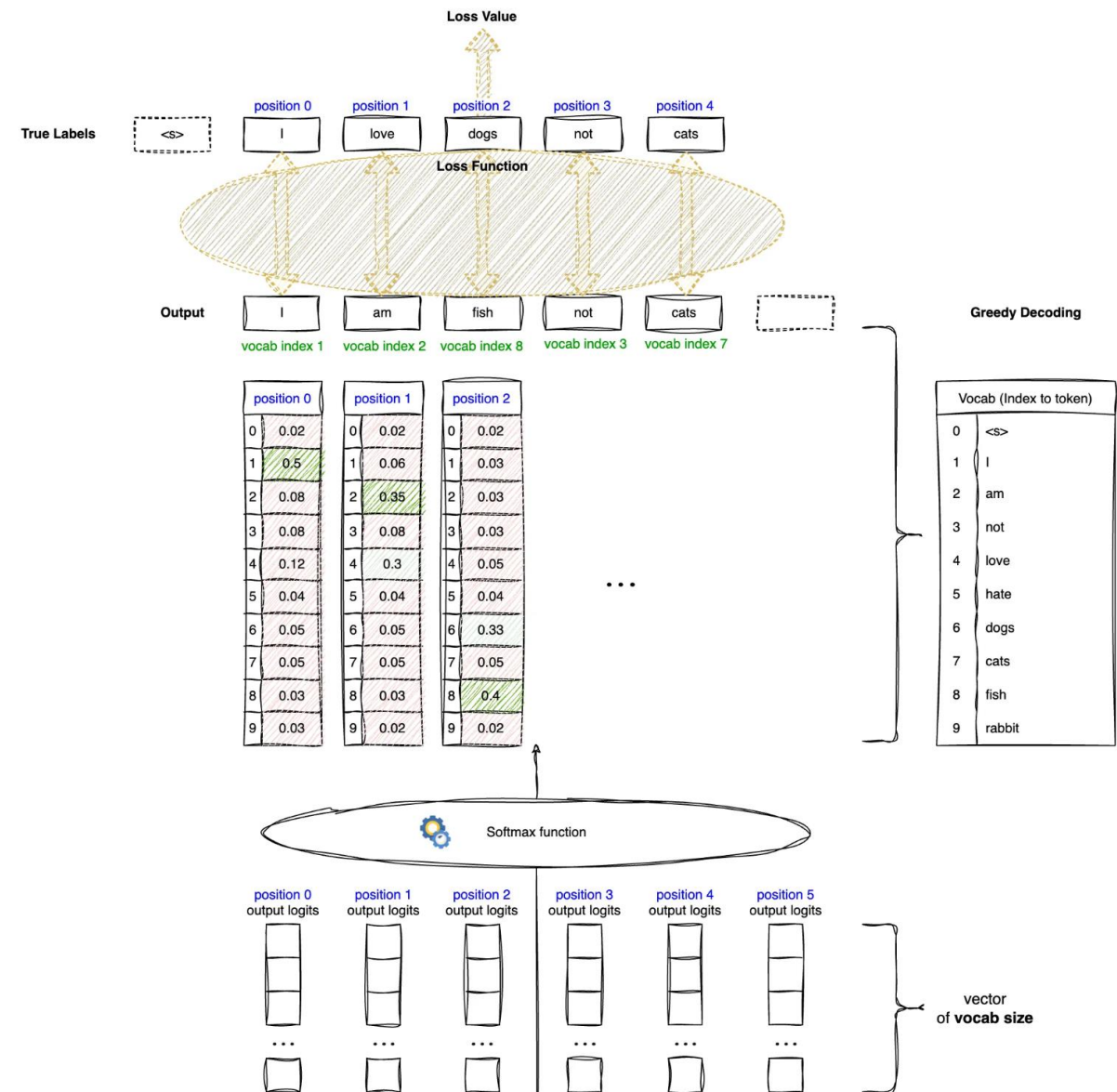
[1] Picture from <https://huyenchip.com/2023/05/02/r1hf.html>

Casual Language Modeling

1.



2.

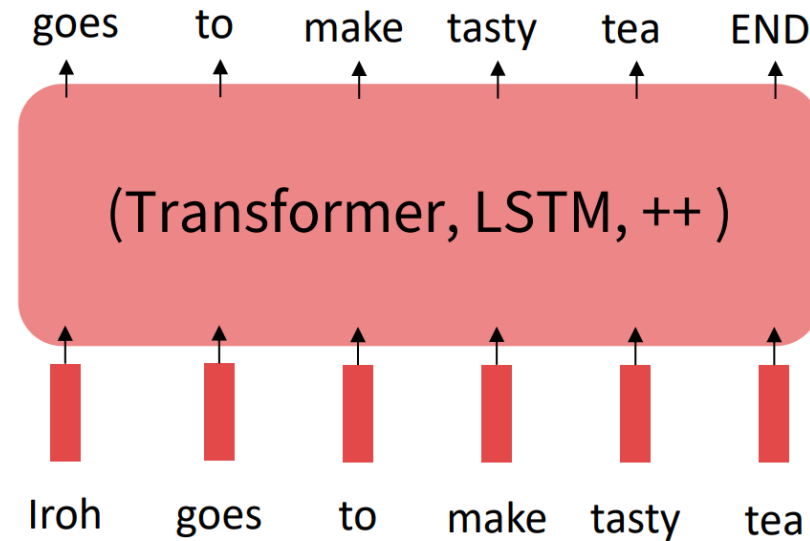


[1] Pictures from <https://medium.com/@jeraldteokj/visualising-loss-calculation-in-large-language-models-1af410a9d73d>

Pre-train

- Language modelling task: $p_{\theta}(w_t|w_{1:t-1})$
- Long-long path: n(skip)-gram models $\rightarrow$ RNN $\rightarrow$ LLM
- LLMs are based on Transformer (decoder) architecture, which predicts the next token for every position
- Loss function? Cross-entropy
- Data? Will cover later in this presentation
- 54 days for Llama3 405B on 16k H100, 400+ interruptions

$$H(p, q) = - \sum_{i=1}^n p(x_i) \log_b q(x_i)$$



[batch_size, context_length, vocab_size]

```
shift_logits = logits[:, :-1, :].contiguous()
shift_labels = tokens[:, 1:]
loss_fct = nn.CrossEntropyLoss()
loss = loss_fct(
    shift_logits.view(-1, shift_logits.size(-1)),
    shift_labels.view(-1)
)
```

Pre-train: power laws

- Model performance depends most strongly on scale, which consists of three factors:
 - the number of model parameters N (excluding embeddings)
 - the size of the dataset D
 - the amount of compute C used for training
- “We expect that larger language models will perform better and be more sample efficient than current models”
- “...every time we increase the model size 8x, we only need to increase the data by roughly 5x to avoid a penalty...”

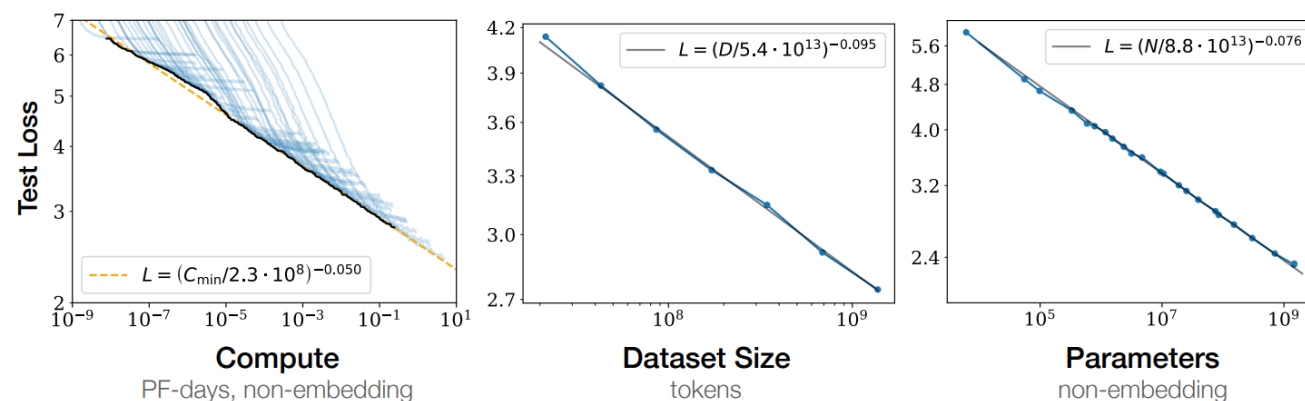


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute² used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

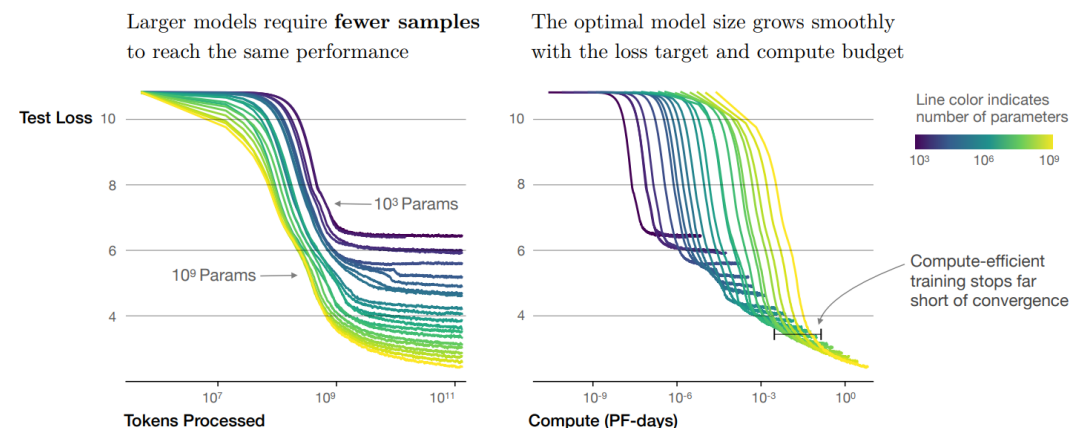


Figure 2 We show a series of language model training runs, with models ranging in size from 10³ to 10⁹ parameters (excluding embeddings).

[1] Picture from Kaplan, Jared, et al. "Scaling laws for neural language models." *arXiv preprint arXiv:2001.08361* (2020).

In context learning?

- Having a pretrained model, prompt engineering magic (e.g. chain-of-thought, CoT) can make things better, but
- Issues with the context
- Usually can't help with the complex tasks

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.



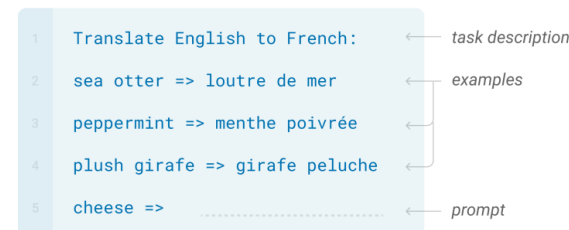
One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.



Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.



Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.

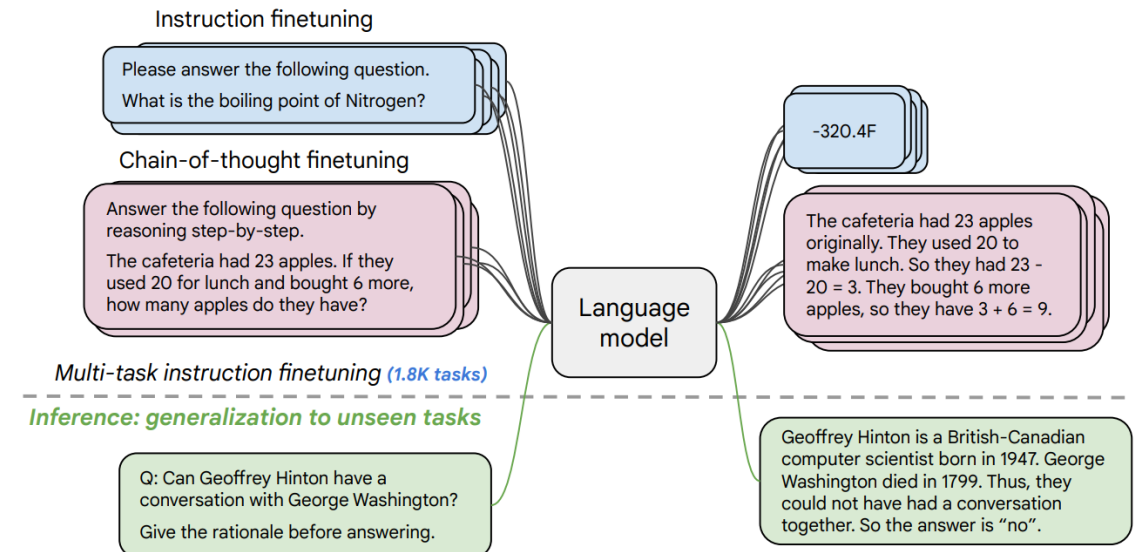


[1] Picture from Brown, Tom B. "Language models are few-shot learners." *arXiv preprint arXiv:2005.14165* (2020).

Alignment: supervised fine-tuning

- Pretraining is actually sequence completion. Not what the final user really wants from LLM
- Let's show it what we really need!
- Supervised fine-tuning (SFT) lets you get more out of the models:
 1. Higher quality results than prompting
 2. Ability to train on more examples than can fit in a prompt
 3. Token savings due to shorter prompts
- SFT is not much different from language model pretraining—both pretraining and SFT use next token prediction as their underlying training objective (only for LLM response, prompt tokens are masked)!

PROMPT	Explain the moon landing to a 6 year old in a few sentences.
COMPLETION	GPT-3
	Explain the theory of gravity to a 6 year old.
	Explain the theory of relativity to a 6 year old in a few sentences.
	Explain the big bang theory to a 6 year old.
	Explain evolution to a 6 year old.



[1] Picture (top) from Ouyang, Long, et al. "Training language models to follow instructions with human feedback." *Advances in neural information processing systems* 35 (2022)

[2] Picture from Chung, Hyung Won, et al. "Scaling instruction-finetuned language models." *Journal of Machine Learning Research* 25.70 (2024)

Alignment: supervised fine-tuning issues

- Acquiring new knowledge via supervised fine-tuning is correlated with hallucinations w.r.t. pre-existing knowledge
- LLMs struggle to integrate new knowledge through fine-tuning and mostly learn to use their pre-existing knowledge
- Limitations: experiments were conducted using a single LLM, and thus it is unclear whether results will vary with different LLMs [1]
- LIMA [2]: “We define the Superficial Alignment Hypothesis: A model’s knowledge and capabilities are learnt almost entirely during pretraining, while alignment teaches it which subdistribution of formats should be used when interacting with users”.

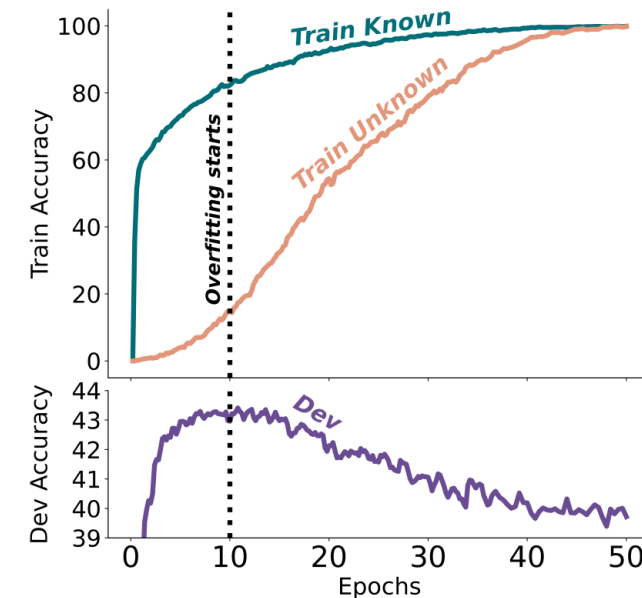


Figure 1: Train and development accuracies as a function of the fine-tuning duration, when fine-tuning on 50% Known and 50% Unknown examples. Unknown examples are fitted substantially slower than Known. The best development performance is obtained when the LLM fits the majority of the Known training examples but only few of the Unknown ones. From this point, fitting Unknown examples reduces the performance.

[1] Gekhman, Zorik, et al. "Does Fine-Tuning LLMs on New Knowledge Encourage Hallucinations?." *arXiv preprint arXiv:2405.05904* (2024).

[2] Zhou, Chunting, et al. "Lima: Less is more for alignment." *Advances in Neural Information Processing Systems* 36 (2024).

Alignment: optimize towards human preferences

- SFT data tells the model what responses are plausible for a given context, but doesn't tell the model how good or how bad a response is
- Lets change this by further training the model using data that represents human preferences:
- prompt | win_response | lose_response
- But how exactly does it use this data?

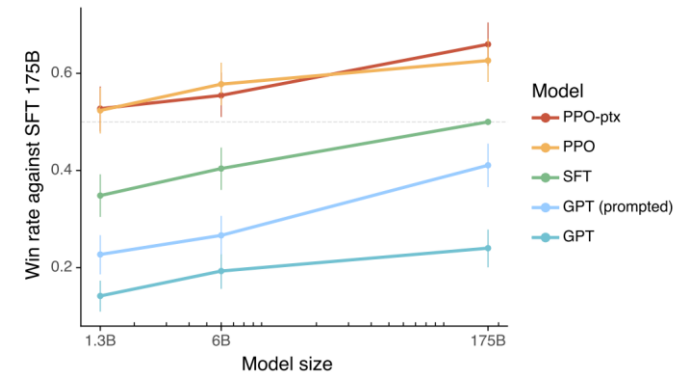
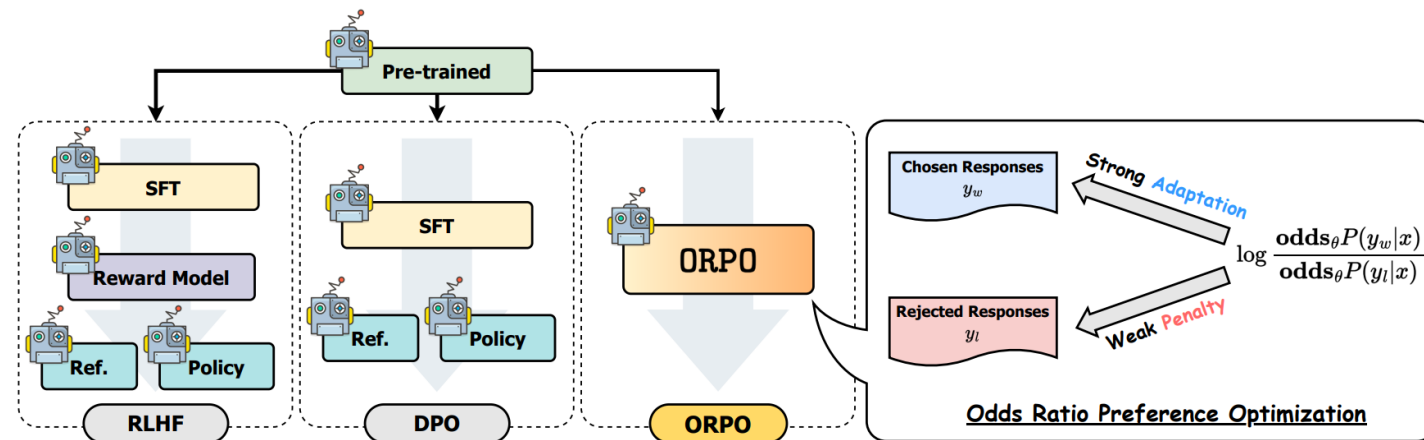


Figure 1: Human evaluations of various models on our API prompt distribution, evaluated by how often outputs from each model were preferred to those from the 175B SFT model. Our InstructGPT models (PPO-ptx) as well as its variant trained without pretraining mix (PPO) significantly outperform the GPT-3 baselines (GPT, GPT prompted); outputs from our 1.3B PPO-ptx model are preferred to those from the 175B GPT-3. Error bars throughout the paper are 95% confidence intervals.

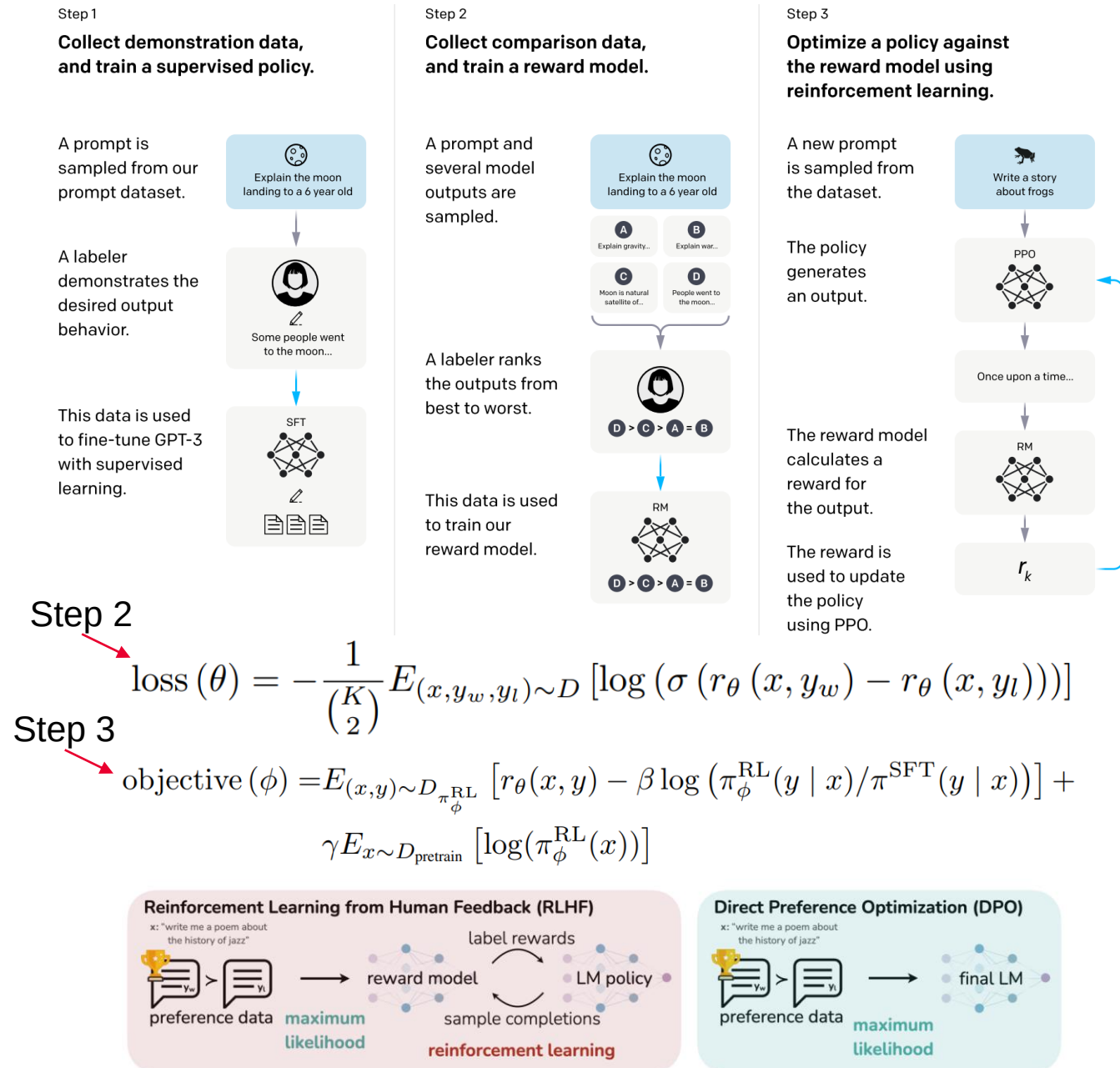


[1] Picture (top) from Ouyang, Long, et al. "Training language models to follow instructions with human feedback." *Advances in neural information processing systems* 35 (2022)
[2] Picture (bottom) from Hong, Jiwoo, Noah Lee, and James Thorne. "Reference-free monolithic preference optimization with odds ratio." arXiv preprint arXiv:2403.07691 (2024)

Alignment: RLHF / DPO

- Reward model takes prompt + response and provide a score
- Reward model can be the copy of SFT model or smaller one trained from scratch
- Train the reward model on curated dataset
- Once it is done, SFT model is trained via Proximal Policy Optimization (PPO) algorithm
- Very effective when tuned well, computationally expensive and tricky to get right
- Direct Preference Optimization (DPO): optimize LM parameters directly on preference data by solving a binary classification problem

[1] Picture (top) from Ouyang, Long, et al. "Training language models to follow instructions with human feedback." *Advances in neural information processing systems* 35 (2022)



Data and Tokenization

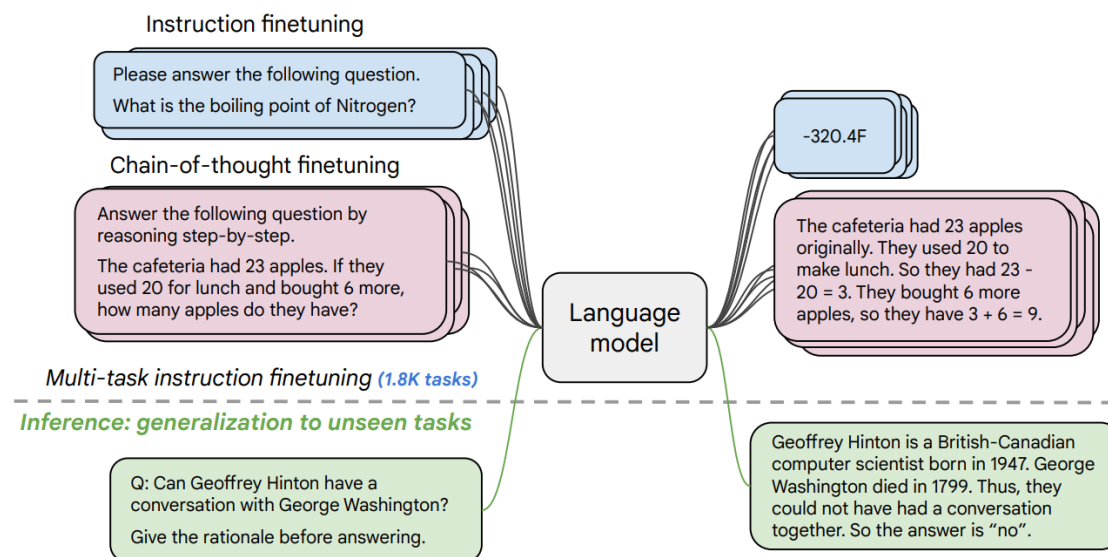


Data

- Let's try to cover 2 questions: from where and how much?
- Different stages – different datasets
- Pre-train: colossal web crawled datasets, books libraries, wiki
- SFT: “demonstration data” generated by humans (crowdsourcing via Upwork/ScaleAI/Toloka/etc), accurately filtered data from pretrain datasets, or even GPT-4 distillation

Model	Training Data
BERT	BookCorpus, English Wikipedia
GPT-1	BookCorpus
GPT-3	CommonCrawl, WebText, English Wikipedia, and 2 book databases (“Books 1” and “Books 2”)
GPT-3.5+	Undisclosed

45TB!



Common Crawl

- Common Crawl maintains a free, open repository of web crawl data that can be used by anyone (from 2008)
- Stats: <https://commoncrawl.github.io/cc-crawl-statistics/>
- CCBot is a Nutch-based (Apache Nutch [1]) web crawler that makes use of the Apache Hadoop project
- Big data for sure (petabytes), This September's crawl contains 2.8B web pages (410TiB)!
- All the crawlers (CC, OpenAI, Google, etc.) can be ~~almost~~ completely disabled via creation of "robots.txt" file [2]



If you want to let GPTBot access certain parts of your website and not others, you'd enter in some code like the following:

```
User-agent: GPTBot
Allow: /directory-1/
Disallow: /directory-2/
```

[1] <https://nutch.apache.org/>

[2] robots.txt: <https://developers.google.com/search/docs/crawling-indexing/robots/intro>

Example of Common Crawl filtering

- C4, Collosal Clean Crawled Corpus. 156B tokens.
- RefinedWeb [1]. Falcon 40B, 5T tokens, 600B publicly available
- RedPajamas-v1. 1.2T
- RedPajamas-v2 [2]. 30T
- Common corpus

[1] Penedo, Guilherme, et al. "The RefinedWeb dataset for Falcon LLM: outperforming curated corpora with web data, and web data only." *arXiv preprint arXiv:2306.01116* (2023)

[2] RedPajamas-v2:
<https://www.together.ai/blog/redpajama-data-v2>

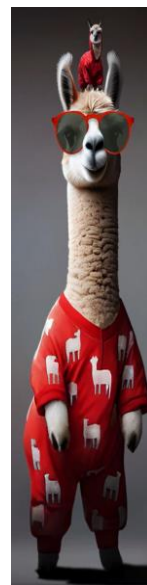
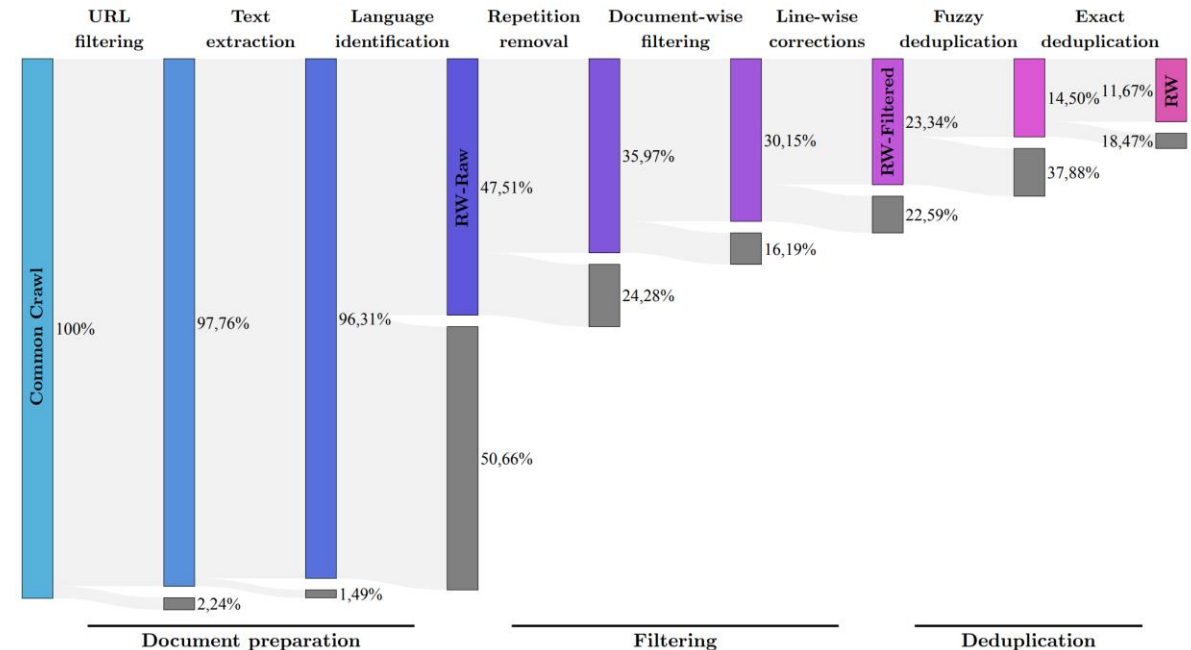


Table 2. Macrodata Refinement aggregates best practices from the state-of-the-art and novel approaches (URL scoring, line-wise filtering, etc.) to produce high-quality web data. On deduplication, we note that MDR is unique in both the scale at which it is performed, and in applying subsequently fuzzy and exact substring methods to improve coverage and scalability.

DOCUMENT PREPARATION			FILTERING		DEDUPLICATION	
URL filtering	Text extraction	Language identification	Document-wise filtering	Line-wise filtering	Deduplication	URL deduplication
Aggregated block-list, URL scoring, common HQ sources blocked Appendix G.1	From WARC using warcio, trafilatura for extraction Barbaresi (2021)	fastText classifier from CCNet, thresholding on top language score Wenzek et al. (2020)	In-document repetition removal and quality heuristics from MassiveWeb Rae et al. (2021)	Remove undesirable lines (call to actions, navigation buttons, social counters, etc.) Appendix G.2	Fuzzy deduplication w/ MinHash + exact substring deduplication w/ suffix arrays Lee et al. (2022)	Remove URLs revisited across Common-Crawl dumps Section 3.3



Data to Model parameters ratio

- 2020—2022, the trend so far in large language model training has been to increase the model size, often without increasing the number of training tokens
- Chinchilla [1] paper: By training over 400 language models ranging from 70 million to over 16 billion parameters on 5 to 500 billion tokens, we find that ... *for every doubling of model size the number of training tokens should also be doubled*
- Authors presented three different approaches to answer the question: Given a fixed FLOPs budget, how should one trade-off model size and the number of training tokens?

[1] Hoffmann, Jordan, et al. "Training compute-optimal large language models." *arXiv preprint arXiv:2203.15556* (2022).

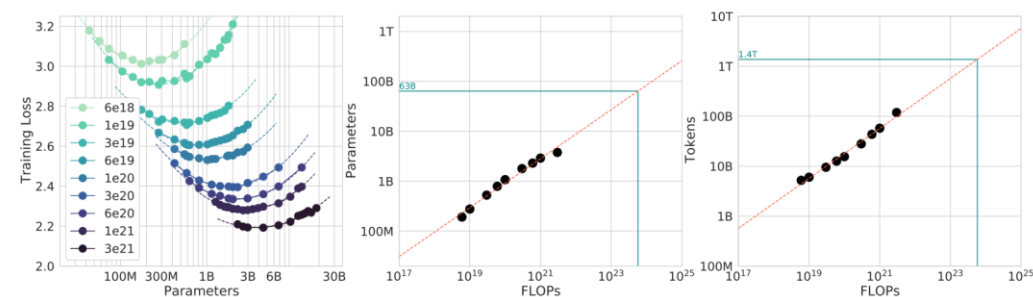


Figure 3 | **IsoFLOP curves.** For various model sizes, we choose the number of training tokens such that the final FLOPs is a constant. The cosine cycle length is set to match the target FLOP count. We find a clear valley in loss, meaning that for a given FLOP budget there is an optimal model to train (**left**). Using the location of these valleys, we project optimal model size and number of tokens for larger models (**center** and **right**). In green, we show the estimated number of parameters and tokens for an *optimal* model trained with the compute budget of *Gopher*.

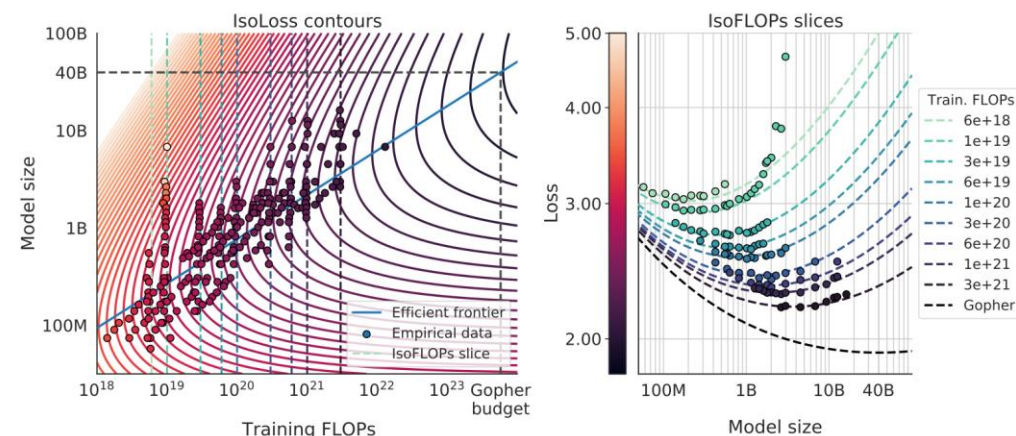


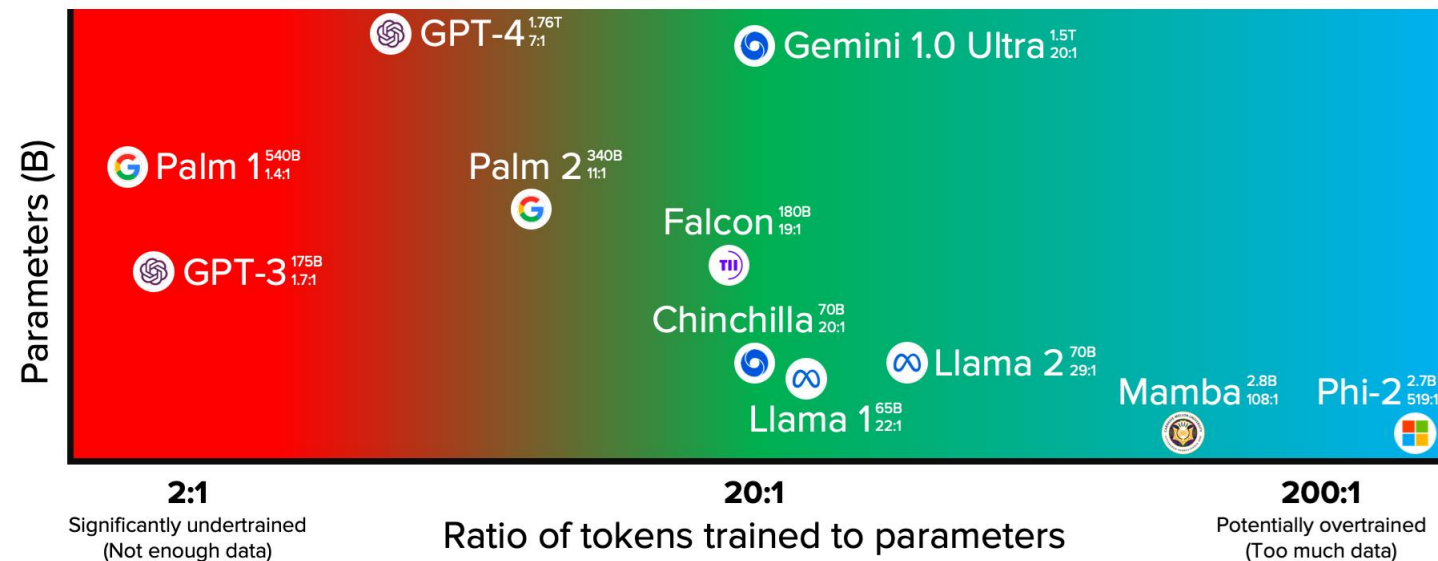
Figure 4 | **Parametric fit.** We fit a parametric modelling of the loss $\hat{L}(N, D)$ and display contour (**left**) and isoFLOP slices (**right**). For each isoFLOP slice, we include a corresponding dashed line in the left plot. In the left plot, we show the efficient frontier in blue, which is a line in log-log space. Specifically, the curve goes through each iso-loss contour at the point with the fewest FLOPs. We project the optimal model size given the *Gopher* FLOP budget to be 40B parameters.

Data to Model parameters ratio II

Table 3 | **Estimated optimal training FLOPs and training tokens for various model sizes.** For various model sizes, we show the projections from Approach 1 of how many FLOPs and training tokens would be needed to train compute-optimal models. The estimates for Approach 2 & 3 are similar (shown in [Section D.3](#))

Parameters	FLOPs	FLOPs (in <i>Gopher</i> unit)	Tokens
400 Million	1.92e+19	1/29, 968	8.0 Billion
1 Billion	1.21e+20	1/4, 761	20.2 Billion
10 Billion	1.23e+22	1/46	205.1 Billion
67 Billion	5.76e+23	1	1.5 Trillion
175 Billion	3.85e+24	6.7	3.7 Trillion
280 Billion	9.90e+24	17.2	5.9 Trillion
520 Billion	3.43e+25	59.5	11.0 Trillion
1 Trillion	1.27e+26	221.3	21.2 Trillion
10 Trillion	1.30e+28	22515.9	216.2 Trillion

Model size (params)	Training tokens (round)	Training data used (estimate)	How much data is that? If 1 book is about 500KB of text (estimate)
Chinchilla/			More books than in...²³
70B	1.4 Trillion	2.3TB	<i>The Kindle store on Amazon US (6.4M).</i>
250B	5 Trillion	8.3TB	<i>All 30 libraries at Yale University (16.6M).</i>
500B	10 Trillion	16.6TB	<i>The Google Books collection (33.2M).</i>
1T	20 Trillion	33.3TB	<i>The US Library of Congress (66.6M).</i>
10T	200 Trillion	333TB	<i>All US public libraries combined (666M).</i>
100T	2 Quadrillion	3.3PB	<i>All bibles ever sold worldwide (6.6B).</i>
250T	5 Quadrillion	8.3PB	<i>A stack all the way to the Moon (16.6B).</i>
500T	10 Quadrillion	16.6PB	<i>4 books about every living human (33.2B).</i>



[1] Table (right) and picture from <https://lilearchitect.ai/chinchilla/>

SFT data

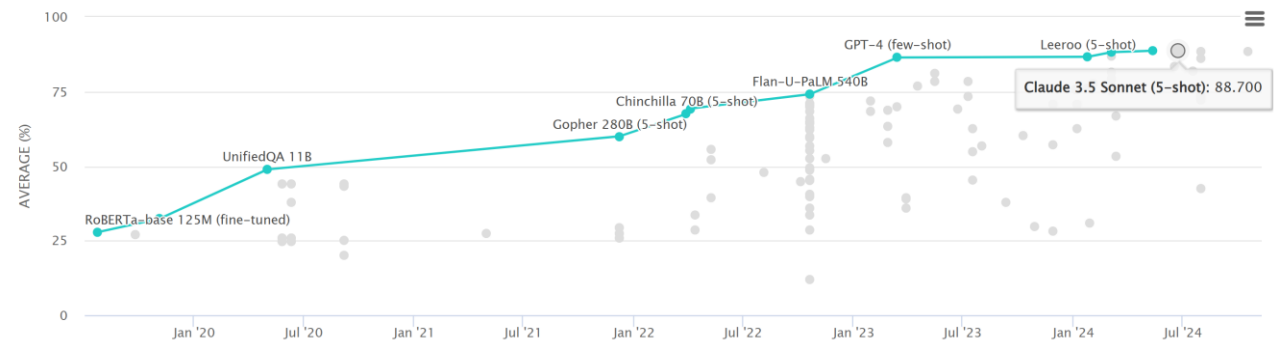
- Collect examples of (instruction, output) pairs across many tasks and finetune an LM. Then evaluate on unseen tasks
- Prior literature has shown that increasing the number of tasks in finetuning with instructions improves generalization to unseen tasks [1, 2]
- SuperNaturalInstructions: 1.6k tasks, no more than 6.5k examples, 3+ million examples totally [1]



MMLU

avg 5-shot

-	Random	25.0
-	Average human rater	34.5
May 2020	GPT-3 5-shot	43.9
Mar. 2022	Chinchilla 5-shot	67.6
Apr. 2022	PaLM 5-shot	69.3
Oct. 2022	Flan-PaLM 5-shot	72.2
	Flan-PaLM 5-shot: CoT + SC	75.2
-	Average human expert	89.8
	Jun. 2023 forecast (Hypermind)	73.2
	Jun. 2024 forecast (Hypermind)	75.0
	Jun. 2023 forecast (Metaculus)	82.7
	Jun. 2024 forecast (Metaculus)	87.6



[1] Wang, Yizhong, et al. "Super-naturalinstructions: Generalization via declarative instructions on 1600+ nlp tasks." *arXiv preprint arXiv:2204.07705* (2022).

[2] Chung, Hyung Won, et al. "Scaling instruction-finetuned language models." *Journal of Machine Learning Research* 25.70 (2024): 1-53.

SFT data quantity?

- LIMA [1]: “We show that fine-tuning a strong pretrained language model on 1,000 carefully curated examples can produce remarkable, competitive results on a wide range of prompts”
- 65B Llama model
- Main limitations: hard to scale & expensive++, not really robust in comparison with production-grade models

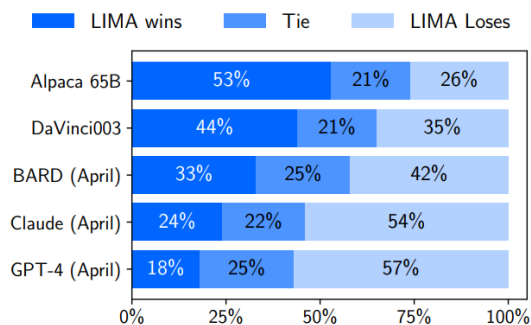


Figure 1: Human preference evaluation, comparing LIMA to 5 different baselines across 300 test prompts.

Source	#Examples	Avg Input Len.	Avg Output Len.
Training			
Stack Exchange (STEM)	200	117	523
Stack Exchange (Other)	200	119	530
wikiHow	200	12	1,811
Pushshift r/WritingPrompts	150	34	274
Natural Instructions	50	236	92
Paper Authors (Group A)	200	40	334
Dev			
Paper Authors (Group A)	50	36	N/A
Test			
Pushshift r/AskReddit	70	30	N/A
Paper Authors (Group B)	230	31	N/A

Table 1: Sources of training prompts (inputs) and responses (outputs), and test prompts. The total amount of training data is roughly 750,000 tokens, split over exactly 1,000 sequences.

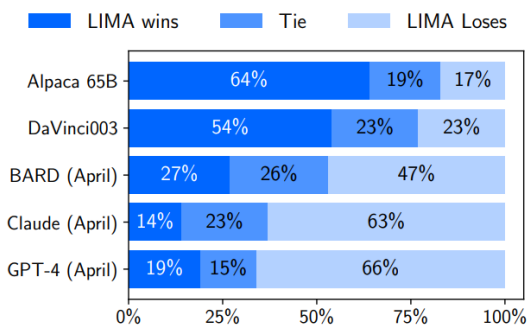


Figure 2: Preference evaluation using GPT-4 as the annotator, given the same instructions provided to humans.

Additional SFT data approaches: MAmmoTH2

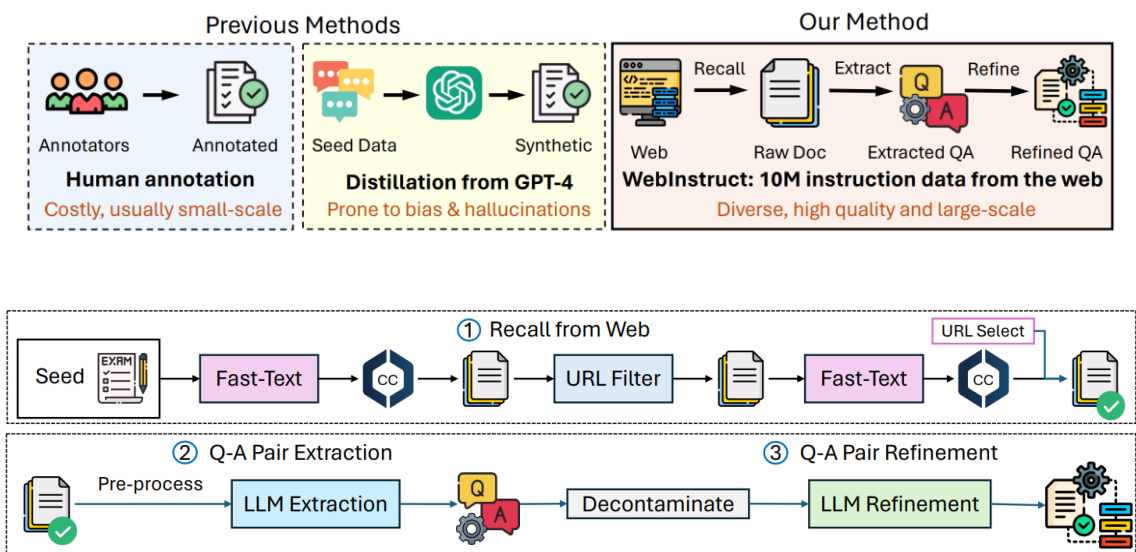


Figure 3: Step 1: Recall relevant documents from Common Crawl. Step 2: Extracting Q-A pairs. Step 3: Refine with the extracted Q-A pairs.

The figure shows the transformation of raw web documents into refined Q-A pairs:

- Raw Docs:** Unformatted Text, Site Information, Ads. Example text includes topics like Science, Anatomy, and a question about simplifying mathematical expressions.
- Extracted QA:** Formatted QA but lacking detailed solutions. The question and answer are extracted from the raw text.
- Rewritten QA:** Formatted QA augmented with detailed solutions. The answer is refined to include a step-by-step explanation of the mathematical simplification process.

Figure 4: An illustrating example from WEBINSTRUCT for the extraction and refinement step.

[1] Yue, Xiang, et al. "Mammoth2: Scaling instructions from the web." *arXiv preprint arXiv:2405.03548* (2024)

Tokenization

- The “tokenization” is a process of text splitting into words or subwords, which then are converted to ids through a look-up table (vocabulary)
- Subword tokenization allows the model to have a reasonable vocabulary size while being able to learn meaningful context-independent representations
- In addition, subword tokenization enables the model to process words it has never seen before, by decomposing them into known subwords

The “tokenization” is a process of text splitting into words or subwords, which then are converted to ids through a look-up table (vocabulary)

	word		vocab mapping	embedding
Common words	hat	→	hat	■
	learn	→	learn	■
Variations	taaaaasty	→	taa## aaa## sty	■ ■ ■
misspellings	laern	→	la## ern##	■ ■
novel items	Transformerify	→	Transformer## ify	■ ■

[1] Tiktokenizer: <https://tiktokenizer.vercel.app/?model=gpt-4o>

[2] cs224n, pretrain lecture: <https://web.stanford.edu/class/cs224n/slides/cs224n-spr2024-lecture09-pretraining-updated.pdf>

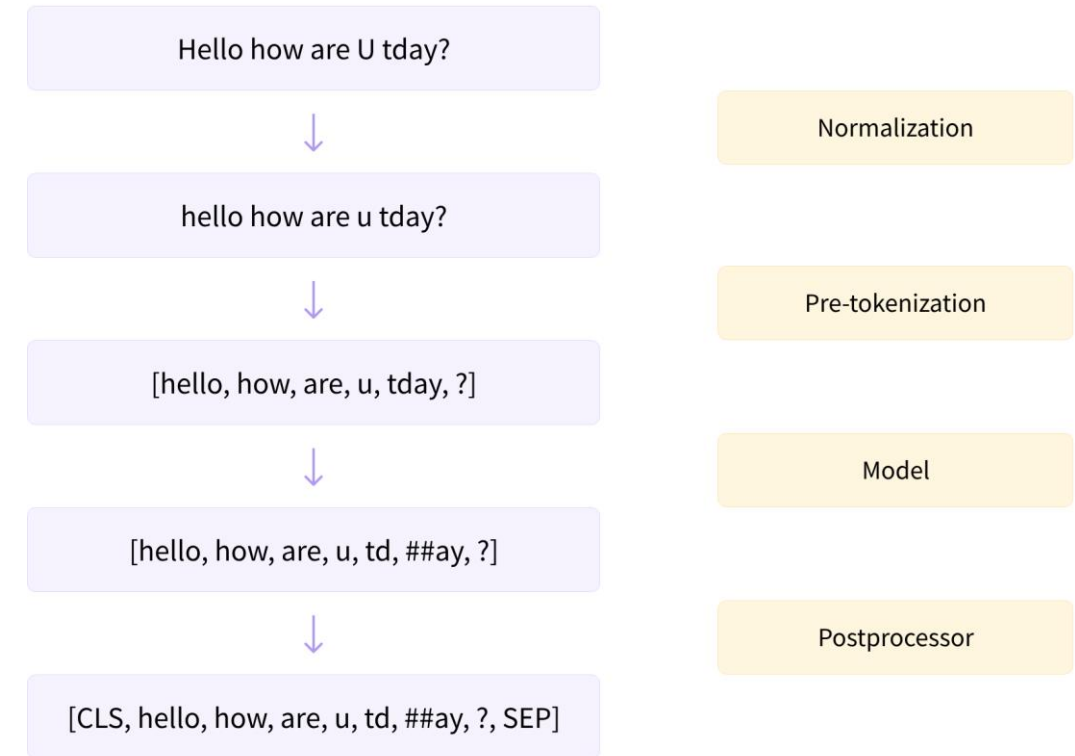
Tokenization pipeline

1. The normalization step involves some general cleanup, such as removing needless whitespace, lowercasing, and/or removing accents, Unicode normalization [1] and others
2. Pre-tokenization splits the text into small entities, e.g. words
3. Tokenization itself
4. Postprocessing. E.g. inserting special tokens like SEP or CLS.

Pre-tokenization example:

"Hello, how are you?" ->

[('Hello', (0, 5)), (',', (5, 6)), ('how', (7, 10)), ('are', (11, 14)), ('you', (16, 19)), ('?', (19, 20))]



[1] Unicode normalization: <https://www.unicode.org/reports/tr15/>

[2] Picture from <https://huggingface.co/learn/nlp-course/chapter6/4?fw=pt>

GPT-4 pre-tokenization regex [1]

`r"""'(?i:[sdmt]ll|ve|re)|[^\r\n\p{L}\p{N}]?+\p{L}+|\p{N}{1,3}| ?[^\s\p{L}\p{N}]++[\r\n]*\s*[\r\n]|\s+(?!\S)|\s+"""`

1. `'(?i:[sdmt]ll|ve|re)`: extracts the contractions in English
2. `[^\r\n\p{L}\p{N}]?+\p{L}+`: extracts sequence of any characters, letters, and numeric digits except “\r” and “\n”
3. `\p{N}{1,3}`: extracts numbers up to 3 digits (!) as single token
4. `?[^\s\p{L}\p{N}]++[\r\n]*`: extracts the special characters, punctuations, or symbols possibly preceded by space and followed by optional newlines
5. `\s*[\r\n]`: extracts newlines (\n) or carriage returns(\r) preceded by whitespace
6. `\s+(?!\S)`: extracts the trailing whitespace at the end of a line or before a newline; also it doesn't extract any non-whitespace characters
7. `\s+`: extracts any remaining sequence of whitespaces that might not be captured by the previous parts

[1] <https://medium.com/@govindarajpriyanthan/build-and-train-gpt-4-tokenizer-from-scratch-ad90d3af0f11>

Byte-pair encoding (BPE)

- Byte pair encoding is a way of converting text into tokens. It has a couple desirable properties:
 1. It's reversible and lossless
 2. It works on arbitrary text, even text that is not in the tokenizer's training data
 3. It compresses the text: the token sequence is shorter than the bytes corresponding to the original text
 4. It attempts to let the model see common subwords

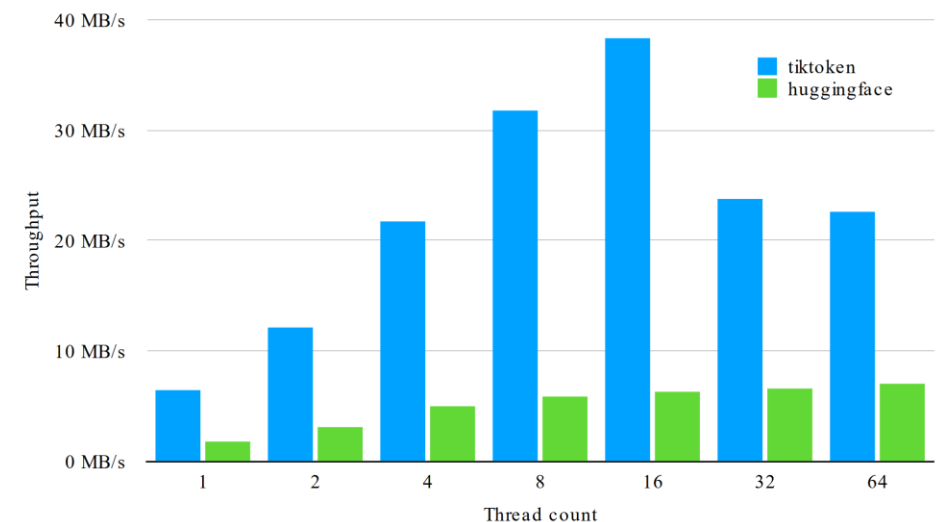
Example:

1. “**aa**abd**aa**abac” => {Z = “aa”} + {“a”, “b”, “c”, “d”}
2. “Z**ab**dZ**ab**ac” => {Y = “ab”, Z = “aa”}
3. “**ZY**d**ZY**ac” => {X = “ZY”, Y = “ab”, Z = “aa”}
4. “XdXac”

[1] Official OpenAI tokenizer: <https://github.com/openai/tiktoken>

- Algorithm:
 1. Start with a vocabulary containing only characters and an “end-of-word” symbol
 2. Using a corpus of text, find the most common adjacent characters and add them as a subword
 3. Replace instances of the character pair with the new subword; repeat until desired vocab size
- Question to the audience: How to deal with tons of Unicode characters?

tiktoken is between 3-6x faster than a comparable open source tokenizer:



BPE alternatives

1. WordPiece [1]

- Same idea as BPE, but instead of finding the most frequent pair, it chose the one that maximizes the likelihood of training data when added to the vocabulary

2. Unigram

- It initializes base vocabulary to a large number of symbols and progressively trims down each symbol to obtain a smaller vocabulary
- The base vocabulary could for instance correspond to all pre-tokenized words and the most common substrings
- It uses likelihood loss over current vocabulary, and the process is as follows: remove XX% of tokens which removal lead to lowest loss increase

3. SentencePiece [2]

- All tokenization algorithms described so far have the same problem: it is assumed that the input text uses spaces to separate words
- Not all languages use spaces to separate words!
- SentencePiece treats the input as a raw input stream, thus including the space in the set of characters to use
- It then uses the BPE or unigram algorithm to construct the appropriate vocabulary

[1] M. Schuster and K. Nakajima, "Japanese and Korean voice search," *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Kyoto, Japan, 2012, pp. 5149-5152

[2] Kudo, T. "Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing." *arXiv preprint arXiv:1808.06226* (2018).

BPE alternatives II

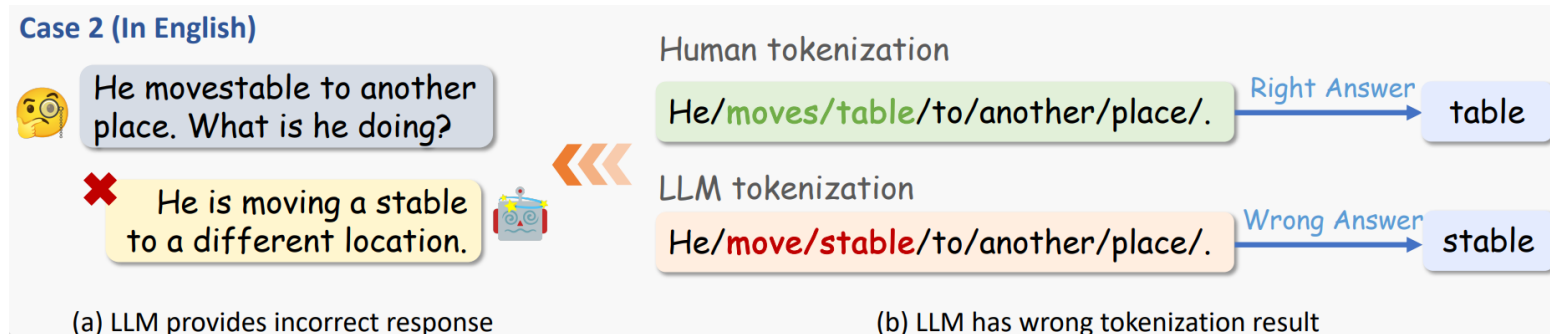
Model	BPE	WordPiece	Unigram
Training	Starts from a small vocabulary and learns rules to merge tokens	Starts from a small vocabulary and learns rules to merge tokens	Starts from a large vocabulary and learns rules to remove tokens (BPE with large vocab size or find most common substrings)
Training step	Merges the tokens corresponding to the most common pair	Merges the tokens corresponding to the pair with the best score based on the frequency of the pair, privileging pairs where each individual token is less frequent: $score = \frac{pair_freq}{first_freq \times second_freq}$	Removes all the tokens in the vocabulary that will minimize the NLL increase computed on the whole corpus
Learns	Merge rules and a vocabulary	Just a vocabulary	A vocabulary with a score for each token
Encoding	Splits a word into characters and applies the merges learned during training	Finds the longest subword starting from the beginning that is in the vocabulary, then does the same for the rest of the word	Finds the most likely split into tokens, using the scores learned during training

[1] Picture from <https://huggingface.co/learn/nlp-course/chapter6/4?fw=pt>

Tokenizer issues

1. Spaces
2. Punctuation
3. Typos

- Let's try it ourselves:
Tiktokenizeer



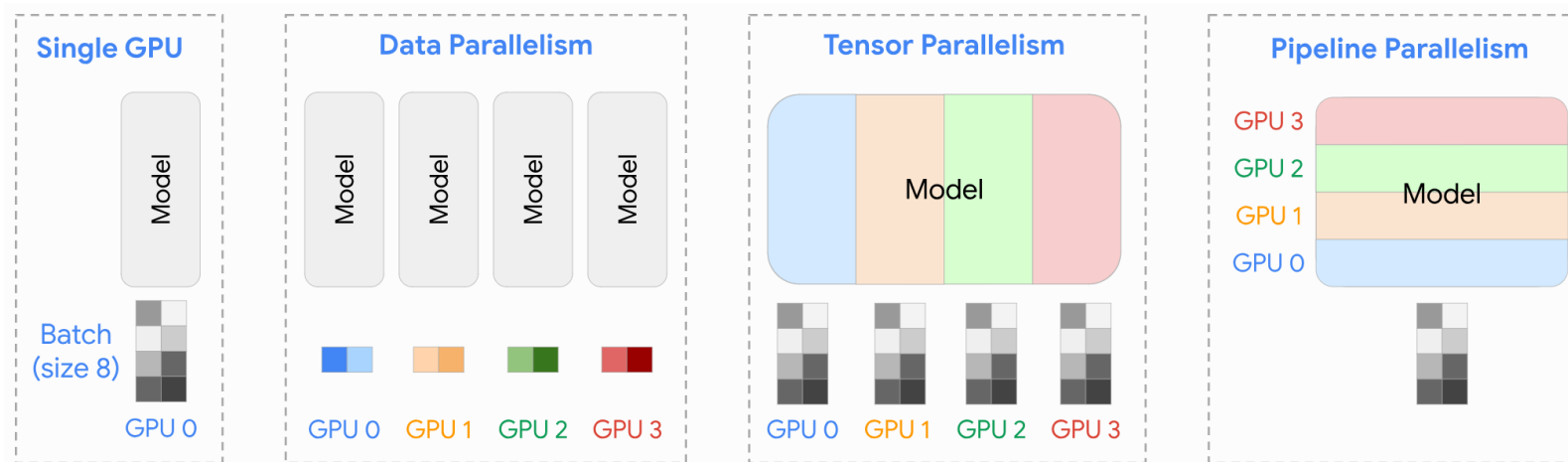
Approach		Origin token	After insertion (Challenging span)	Model tokenization	Human tokenization
Before	Schema Example	BC stable	ABC movestable	[A, BC] [move, stable]	[AB, C] [moves, table]
After	Schema Example	AB driven	ABC drivenothing	[AB, C] [driven, othing]	[A, BC] [drive, nothing]
Before & After	Schema Example	BC this	ABCD sixthisland	[A, BC, D] [six, this, land]	[AB, CD] [sixth, island]

Parallelism



Parallelism types

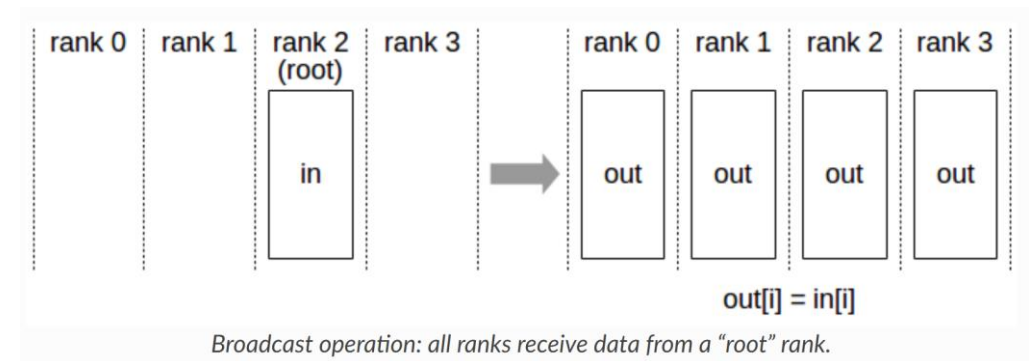
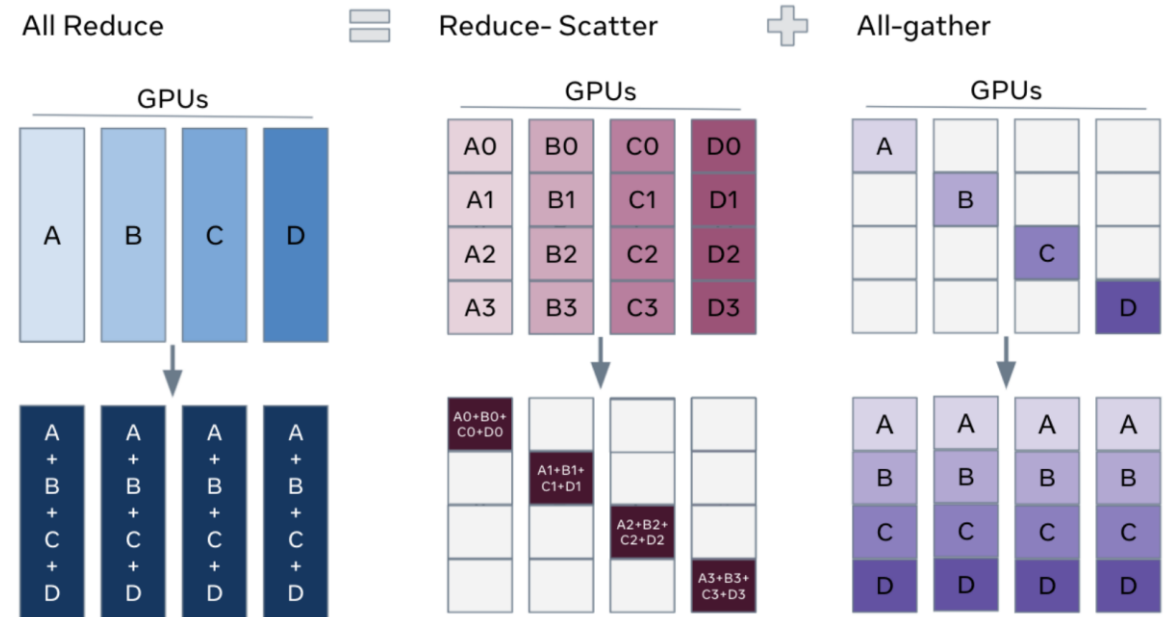
- This won't surprise you: modern LLM are trained using large GPU/TPU/NPU clusters (usually 8 cards) from hundreds to tens of thousands of cards:
 1. model weights usually can't be stored on 1 card
 2. optimizer states (AdamW) takes 12 bytes per parameter (full precision weights copy for mixed precision training, momentum and variance)
 3. trillion tokens to process
- LLM training is indeed a challenge, so engineers came up with a number of techniques to parallel this process:
 1. Data Parallelism (DP)
 2. Tensor Parallelism (TP)
 3. Model/Pipeline Parallelism (MP/PP)
 4. Sequence Parallelism (SP)



[1] Picture from FSDP tutorial: <https://uvadlc-notebooks.readthedocs.io/en/latest/index.html>

Communication operations

- The NVIDIA Collective Communications Library (NCCL, pronounced “Nickel”) is a library providing inter-GPU communication primitives that are topology-aware and can be easily integrated into applications
- NCCL implements both collective communication (*broadcast*, *all gather*, *reduce*, *reduce scatter*, and *all reduce*)
- and point-to-point send/receive primitives
- It is not a complete parallel programming framework, it is a library focused on accelerating inter-GPU communication



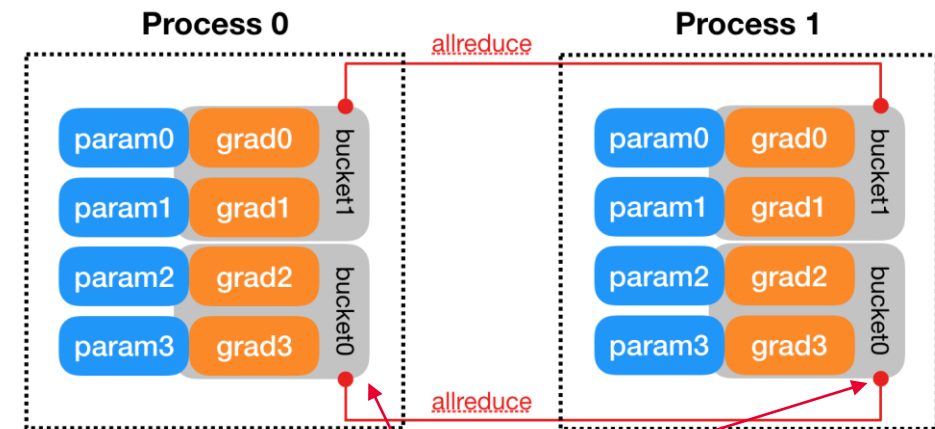
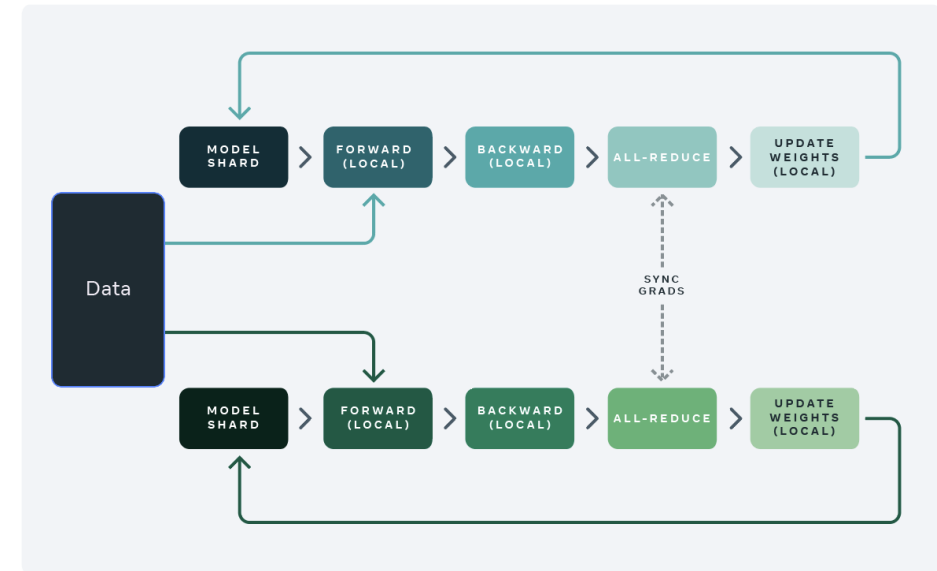
[1] Picture (top) from https://pytorch.org/tutorials/intermediate/FSDP_tutorial.html

[2] Picture (bottom) from <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html>

Data Parallelism

- The (distributed) data parallel idea is quite simple:
 1. each worker (card) within node will have its own copy of the weights
 2. once all the workers finish their computation on mini-batch, all the gradients are all-reduced
 3. and the weights are updated
- DDP works with it as well by using forward graph breaks

Standard data parallel training



Reverse order

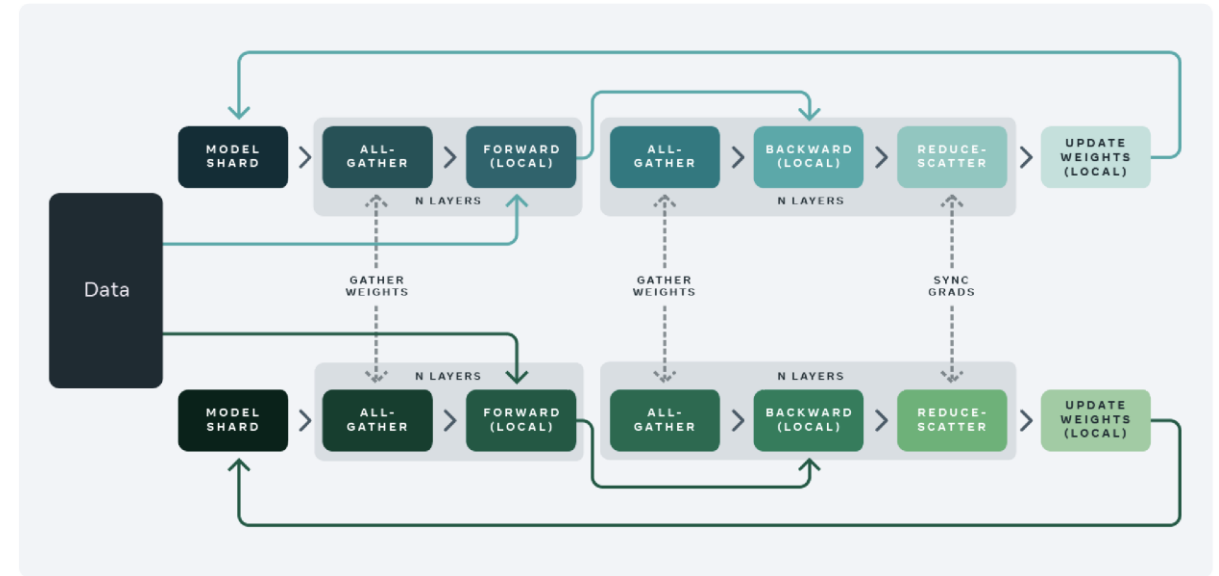
[1] Picture (top) from <https://engineering.fb.com/2021/07/15/open-source/fsdp/>

[2] Picture (bottom) from <https://pytorch.org/docs/stable/notes/ddp.html>

Fully-Sharded Data Parallel

- An alternative extension of vanilla DDP was implemented by Meta and is called FSDP [1]
- FSDP performs the same computation as local training by communicating parameter on-demand before computations
- FSDP has 2 main regimes: *full sharding* and *hybrid sharding*
- Full sharding: each card gets $1/N_d$ model params
- Hybrid sharding occurs when the sharding factor is from 1 to N_d , which brings a trade-off between memory saving and throughput degradation

Fully sharded data parallel training



[1] Zhao, Yanli, et al. "Pytorch fsdp: experiences on scaling fully sharded data parallel." *arXiv preprint arXiv:2304.11277* (2023).

[2] https://pytorch.org/docs/stable/distributed/fsdp/fully_shard.html

[3] Picture from <https://engineering.fb.com/2021/07/15/open-source/fsdp/>

State of the Industry



Tier-1 players



DeepSeek AI

- Great contributor to the Open-source
- Chinese
- Suffers from the Export Control
- V3, R1



OpenAI

- Truly pioneering solutions
- Scandals, a lot
- Looking towards its own robotics, agents and Twitter
- GPT-4o, o1, o3



Anthropic

- Focuses on Safety
- Considered as the best in terms of the coding assistance
- Invests much into LLM interpretability
- Claude Sonnet 3.7



xAI

- Grok-3



Meta AI

- Great contributor to the Open-source
- Looking towards AI that interacts with the environment
- LLaMA-series



Alibaba Cloud

- Great contributor to the Open-source
- Chinese
- Suffers from the Export Control
- Qwen-series



Google DeepMind

- Truly pioneering solutions
- Looking towards its own robotics and neuro-symbolic systems
- Gemini 2.5, AlphaGeometry

Challenges

- Open-source vs. Closed-source
- Trade wars, Export Control and other bullshit
- Redundant power consumption
- Low academic standards
- What's next?
- Benchmark hacking